

Projet TPI

Share++

Travail pratique individuel
Solution de partage de fichiers

Apprenti : Nathan Filipowitz
Chef de projet : Pascal Benzonana
Experts : Xavier Carrel, Guillaume Blanco

Table des matières

1	Analyse préliminaire.....	3
1.1	Rappel du projet.....	3
1.2	Environnement de développement.....	3
1.3	Objectifs.....	5
1.4	Méthode de gestion de projet.....	6
1.5	Planification initiale.....	7
2	Analyse / Conception.....	7
2.1	Concept.....	7
2.2	Analyse fonctionnelle.....	8
2.3	Stratégie de test.....	10
2.4	Risques techniques.....	12
2.5	Planification.....	12
2.6	Dossier de conception.....	18
3	Réalisation.....	19
3.1	Environnement.....	19
3.1.1	Développement sur laptop :.....	20
3.2	Structure du projet.....	21
3.3	Fonctionnalités techniques importantes.....	22
3.3.1	Workflow Build & Release automatisé.....	22
3.3.2	Logging.....	22
3.3.3	Point d'accès Wi-Fi.....	26
3.3.4	Détection et intégration Tailscale.....	28
3.3.5	Utilisation en ligne de commande (CLI).....	31
3.3.6	Refonte de la page de téléchargement.....	32
3.4	Liste des documents fournis.....	34
3.5	Résultat des tests.....	35
4	Conclusions.....	37
4.1	Objectifs atteints.....	37
4.2	Points positifs.....	38
4.3	Difficultés particulières.....	38
4.4	Expérience d'apprentissage.....	39
4.5	Perspectives d'amélioration.....	39
5	Annexes.....	40
5.1	Résumé du rapport du TPI.....	40
5.2	Sources.....	40
5.3	Glossaire.....	44
5.4	Journal de travail.....	48
5.5	Journal de bord.....	68
5.6	Récapitulatif.....	70
5.7	Plan de tests.....	70
5.8	Journal de tests.....	72
5.9	Archives du projet.....	74

1 Analyse préliminaire

1.1 Rappel du projet

Share++ est une application de bureau Windows compilée avec Flet, un framework Python basé sur Flutter. Elle produit un exécutable autonome ne nécessitant aucune installation de Python ni de navigateur sur la machine cible. L'objectif de l'application est de faciliter le partage de fichiers et de répertoires entre les appareils d'un même réseau local. Elle permet de se passer des services de cloud tiers (OneDrive, WeTransfer, Gmail, etc.) pour les transferts internes, garantissant une confidentialité totale des données et une vitesse de transfert maximale limitée uniquement par le débit du réseau local.

Ce TPI reprend le travail réalisé lors du préTPI et y ajoute cinq axes d'amélioration définis dans le cahier des charges, en plus d'apporter une correction des points laissés en suspend lors du dernier projet. Voici les axes d'améliorations dans les grandes lignes. Ils seront plus amplement détaillés à la section 1.3 :

Connectivité avancée : création d'un point d'accès Wi-Fi ad hoc

Traçabilité et audit : journalisation persistante des téléchargements

Partage à distance : détection et intégration Tailscale

Expérience utilisateur : refonte de l'interface web de téléchargement

Automatisation et CLI : interface en ligne de commande headless

1.2 Environnement de développement

Le développement a été effectué principalement sur le même laptop Linux personnel (Fedora 43), avec l'ordinateur Windows mis à disposition par le CPNV pour tester l'application et faire la validation finale des fonctionnalités. Le dépôt GitHub du préTPI a été réutilisé après archivage de son état dans une branche dédiée « prétpi ».

Détails de l'environnements :

Catégorie	Technologie / Librairie	Version	Rôle et justification
Langage de programmation	Python	3.12	Langage principal, choisi pour sa simplicité de développement et ses nombreuses librairies.
Framework UI	Flet (basé sur Flutter)	0.82.2	Framework de l'interface graphique de l'application, choisi pour son style, sa documentation remplie d'exemples concrets et le développement multiplateforme.
Serveur Web	aiohttp	3.13.3	Serveur HTTP asynchrone permettant de gérer les transferts de fichiers sans bloquer l'interface.
Génération QR Code	qrcode, Pillow	8.2 / 11.3.0	Génération dynamique de QR Codes pour faciliter l'accès à la page de téléchargement depuis un smartphone.

Tests Unitaires	pytest / pytest-asyncio	8.4.2	Tests automatisés pour valider la logique métier.
Packaging	Flet CLI (`flet build`)	0.82.2	Outil de build natif transformant le script Python en exécutable Windows (.exe) avec les dépendances Flutter.
Installeur	Inno Setup	6.7.1	Création d'un assistant d'installation professionnel, les dépendances entre /Program Files et /Appdata et apportant les fonctionnalités des applications windows au programme.
CI / CD	Workflow Github Actions	-	Automatisation du build et du déploiement des versions (Release) à chaque mise à jour du code.
Design UI	Penpot	0.23.1	Création des maquettes de l'interface graphique et de la page de téléchargement
Licence	MIT	-	Licence permissive permettant l'utilisation, la modification et la distribution libre du logiciel.

Justification des choix

Python a été retenu pour sa simplicité, sa grande utilisation dans le cadre des cours, et sa richesse en librairies.

Flet repose sur Flutter comme moteur de rendu natif, ce qui produit un exécutable Windows autonome **sans navigateur embarqué**. L'alternative principale aurait été PyQt ou tkinter, mais Flet offre un style Material Design moderne, cross-platform pour une future extension de la compatibilité sur un seul code source. Sa documentation est également très riche en exemples concrets et la galerie des widgets permet de très facilement se repérer les widgets que l'on voudrait utiliser. L'inconvénient majeur de Flet (son initialisation de Flutter avant Python), empêche un mode CLI dans le même exécutable. Ce problème a été identifié et résolu par la création d'un second exécutable PyInstaller ajouté comme étape du workflow.

aiohttp a été imposé. Les alternatives possibles auraient été Flask ou FastAPI. Aiohttp a l'avantage d'être entièrement asynchrone, permettant de ne jamais bloquer la boucle d'événement Flet. (Une solution synchrone aurait bloqué l'interface graphique pendant les transferts de fichiers).

Inno Setup a été choisi pour produire un installeur Windows professionnel gérant les raccourcis et l'entrée de registre du menu contextuel. Il s'agit du projet d'automatisation de création d'installateurs parmi les plus aboutis existant et supporte nativement la désinstallation du programme.

GitHub Actions gère entièrement le build, le packaging et la publication grâce à un workflow déclenché par tag de version, garantissant que chaque release soit construite dans un environnement propre et identique.

La Definition of Done appliquée à chaque Sprint représentant les principales fonctionnalités: code implémenté et commité, fonctionnalité testée manuellement sur Windows, cas de test automatisé ajouté si applicable, documentation de la section 3.3 mise à jour."

1.3 Objectifs

Les objectifs principaux de Share++ sont basé sur les objectifs présents dans le cahier des charges, mais ont été reformulés comme demandé :

- **Connectivité Avancée : Création d'un Point d'Accès (Acces Point)**
 - Ce module permet de résoudre le problème de la segmentation réseau entre l'hôte et le client mobile. L'hôte créera un réseau wifi temporaire ad-hoc, afin d'intégrer la machine cliente dans le sous-réseau, afin de rendre la page de téléchargement accessible avec le QR Code.
- **Traçabilité et Audit : Historique des Téléchargements**
 - Couche de supervision du fonctionnement de l'application. Système de journalisation (logging) persistant gardant une trace de chaque requêtes réussies (status 200), ainsi que chaque requête échouées. L'historique affiche le nom du fichier, l'horodatage précis, l'adresse IP du client et tente d'identifier le type d'appareil (User-Agent parsing)
- **Partage à Distance : Détection et Intégration Tailscale**
 - L'objectif de cette partie est d'étendre la portée du partage au-delà du réseau local physique (LAN) vers un réseau maillé sécurisé (Overlay Network) par le biais d'une détection automatique de la présence du service Tailscale en cours d'exécution sur la machine hôte. Si actif, l'application récupère l'adresse IP « Tailscale » (généralement commençant par 100.x.x.x) et propose de copier un lien sécurisé accessible par tout autre appareil membre du même « Tailnet », permettant le partage de fichiers à travers Internet sans configuration de redirection de port (NAT traversal).
- **Expérience Utilisateur : Refonte de l'interface web de téléchargement**
 - Le candidat va procéder à une réécriture complète du frontend de la page de téléchargement servie par l'application. L'interface devient « Responsive Design » (adaptée aux mobiles, tablettes et bureaux), intègre une charte graphique moderne et offre une prévisualisation plus claire des fichiers disponibles (icônes selon le type, taille, date).
- **Automatisation et CLI : Interface en ligne de commande**
 - Ouverture de l'application aux utilisateurs avancés et aux scripts d'automatisation sera proposée.
 - Pour se faire, il procédera au développement d'un exécutable en ligne de commande (sharepp.exe) permettant de lancer un serveur headless (sans interface graphique) sur un répertoire spécifique.
 - Syntaxe cible : Sharepp /chemin/vers/dossier – port 3001 – secure
 - Fonctionnalités : le paramètre –secure active une protection par mot de passe (Basif Auth ou page de garde), et le terminal affiche un QR Code généré en ASCII/UTF-8 ainsi que les logs de connexion en temps réel.

Les corrections des défauts présents non réglés du préTPI se feront une fois ces fonctionnalités clés réalisées.

Le serveur génère une page web simple listant les fichiers disponibles, avec des

liens de téléchargement directs. L'application est compilée en exécutable autonome, ne nécessitant aucune installation de Python, ou d'aucune autre dépendance sur la machine cible.

Priorisation des exigences

Les exigences ont été formalisées sous forme d'issues GitHub et organisées dans GitHub Projects. Chaque issue représente une User Story du projet à implémenter et chaque issue reçoit une valeur pour les attributs suivants : "Size" (XS, S, M, L, XL), représentant la durée d'implémentation, "Priority" (P0, P1, P2) pour représenter l'importance de la fonctionnalité dans le projet, "Estimate" (1-10) représente une estimation de la charge de travail. Ajouter pour chaque issue une date de début, date de fin et une itération cible permet d'obtenir un aperçu du travail effectué durant chaque jour. Cette organisation a permis de maintenir une vision claire du backlog tout au long du projet. J'avais donc toujours quelque chose à faire.

Les Sprint ont été organisés par fonctionnalité à ajouter, de la plus conséquente à la plus rapide. Les fonctionnalités qui n'ont pas pu être réalisées dans le délai imparti ont été documentés comme travaux futurs.

1.4 Méthode de gestion de projet

La méthode Scrum a été retenue pour structurer ce projet. Ce choix se justifie principalement par l'imprévisibilité du fonctionnement des technologies entre elles : ce TPI reprend un projet existant dont l'ajout des fonctionnalités comme décrite dans les objectifs n'est pas garanti, notamment concernant le packaging Flet et les contraintes matérielles liées au point d'accès Wi-Fi. Scrum permet de livrer de la valeur par itérations courtes et d'ajuster les priorités lorsqu'un obstacle survient ce qui s'est avéré nécessaire à deux reprises lors du projet.

Il convient toutefois de noter que Scrum est conçu pour des équipes plutôt que pour un projet individuel. Les rôles définis ont du être adapté pour ce cadre là et j'ai donc joué les rôles de développeur, Scrum Master, et M. Benzonana jouait le rôle du Product Owner, lors de nos Sprint Review hebdomadaires. Dans le cadre d'un projet individuel, certains rituels Scrum ont été adaptés : les daily standups ont pris la forme d'une auto-évaluation rapide, et les sprint reviews ont été conduites avec le chef de projet plutôt qu'avec une équipe. La valeur principale de Scrum pour ce projet est donc sa structure en sprints limités dans le temps, avec un Definition of Done explicite et un backlog priorisant les User Stories géré via GitHub Projects.

1.5 Planification initiale

Le projet débute le 23 Avril 2026 et dure jusqu'au 20 Mai 2026. Il est découpé en 4 sprints :

1. Sprint 1 (connectivité, point d'accès, traçabilité)
2. Sprint 2 (Partage à distance et utilisation CLI)
3. Sprint 3 (Expérience utilisateur)
4. Sprint 4 (Correction préTPI et derniers correctifs)

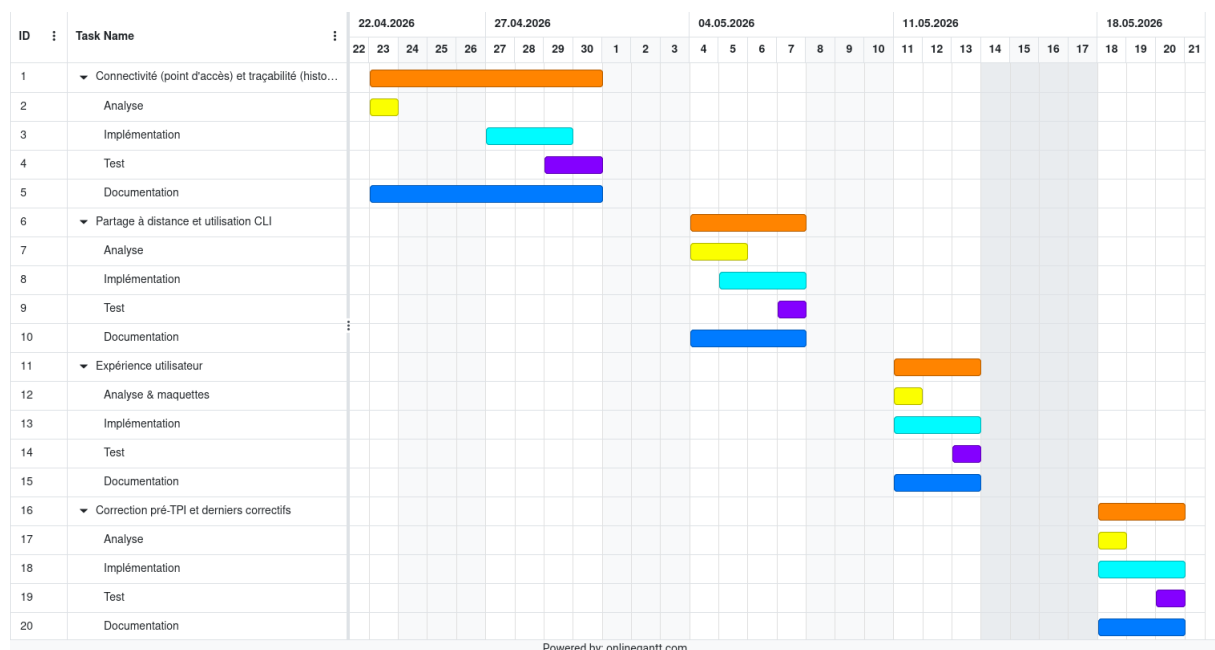


Figure 1: Gantt du planning initial

2 Analyse / Conception

2.1 Concept

Le concept est repris du projet préTPI (Share++ v1.0) et a été étendu pour inclure les nouveaux objectifs.

Le projet repose toujours sur une architecture client-serveur gérée par une application Windows native. Le code suit également toujours le modèle MVC afin de préserver la bonne organisation du code et une future maintenance de l'application :

- **Backend** : Un serveur aiohttp asynchrone gère les requêtes HTTP et le service des fichiers. Le backend sera étendu afin de supporter l'exposition de deux nouvelles interfaces réseau. La première interface est Tailscale
- **Frontend** : Une page html listant les fichiers partagés servie lorsque le partage est actif
- **Persistence** : Les paramètres de l'utilisateur, ainsi que des logs sont persistents et stockés dans des fichiers .json ou .log dédiés.
- **Application** : Une application windows native servant à gérer le serveur et ses paramètres.

2.2 Analyse fonctionnelle

Le cas d'utilisation principal reste inchangé : l'utilisateur sélectionne un dossier local, démarre un serveur de fichier, et les autres appareils y accèdent via un QR code ou une URL afin de télécharger les fichiers partagés. Les nouveaux cas d'utilisation sont les suivants :

- Un utilisateur souhaitant partager des fichiers avec un appareil mobile sans être sur le même réseau peut activer un point d'accès Wi-Fi ad-hoc depuis l'application, ce qui crée un sous-réseau dédié entre l'hôte et le client et permet ainsi d'accéder au partage plus facilement.
- Un utilisateur souhaitant partager des fichiers à distance (hors LAN) et disposant de Tailscale peut utiliser l'adresse Tailscale générée automatiquement par l'application.
- Un utilisateur avancé ou souhaitant automatiser le partage peut utiliser l'interface CLI pour démarrer un serveur headless sans interface graphique.
- L'historique des connexions est conservé dans un fichier de log persistant, consultable après la fermeture de l'application, avec le type d'appareil identifié via le User-Agent.

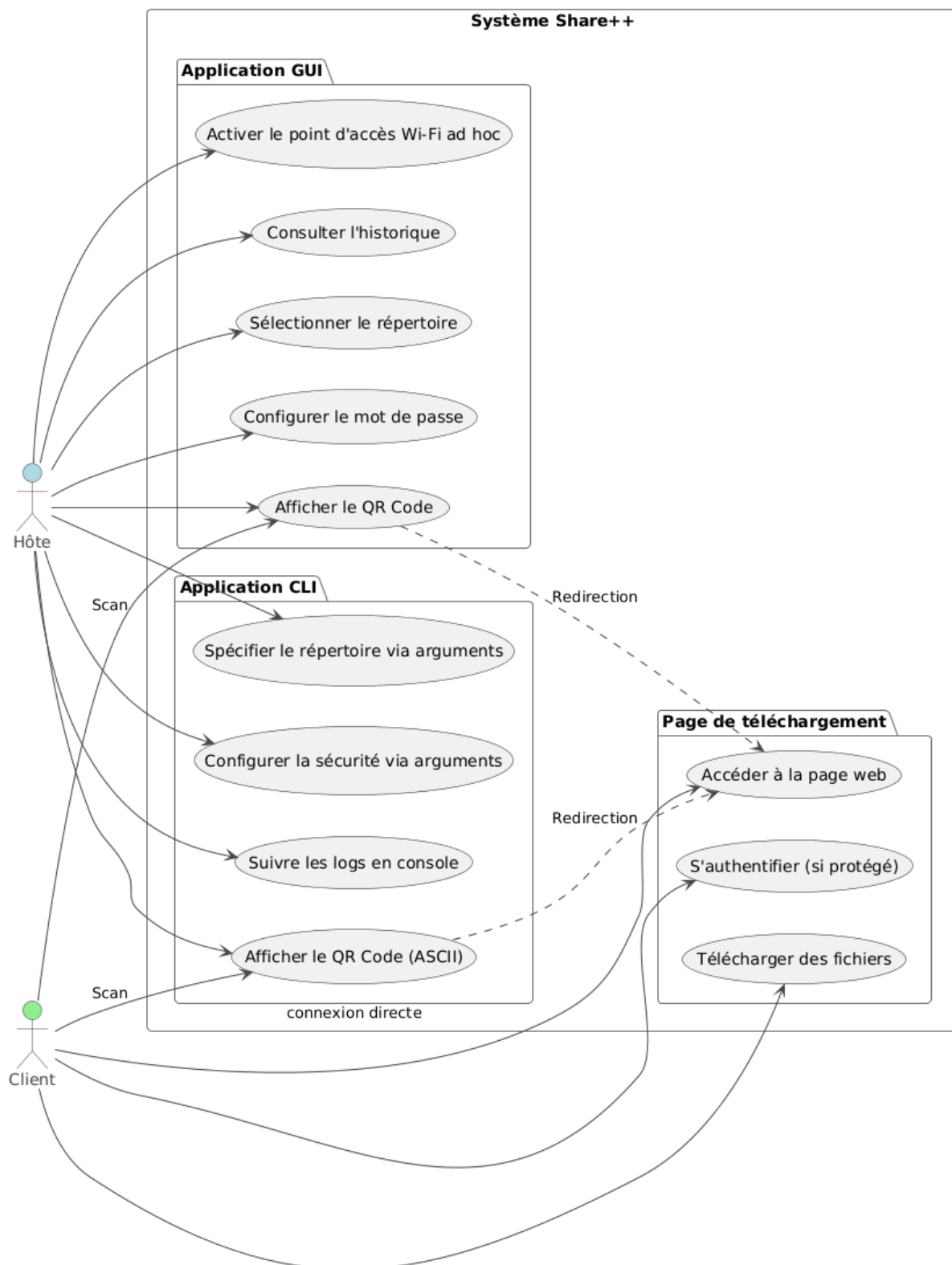


Figure 2: Diagramme des cas d'utilisation du système Share++

J'ai décidé de garder la licence MIT pour ce projet, car il s'agit de la licence la plus permissive au niveau de la transparence du code et sa réutilisation pour autrui.

2.3 Stratégie de test

La stratégie de test vise à valider les nouvelles fonctionnalités tout en s'assurant que les fonctionnalités existantes n'aient pas régressé.

Types de tests :

1. **Tests unitaires** : Vérification de la logique de parsing du User-Agent, de l'écriture et de la rotation des logs dans le fichier `sharepp.log`, et de la détection de Tailscale (présence du service, récupération de l'adresse IP).
2. **Tests d'intégration** : Vérification que le serveur aiohttp expose correctement les fichiers sur les différentes interfaces réseau (LAN, hotspot ad-hoc, Tailscale), et que les logs sont correctement écrits lors des requêtes HTTP.
3. **Tests End-to-End** : Scan du QR code depuis un smartphone connecté au point d'accès ad-hoc créé par l'application, téléchargement de fichiers et de répertoires, et vérification de la traçabilité dans le fichier log.
4. **Tests Utilisateur** : Utilisation de l'application par un tiers, novice en informatique pour valider l'utilisation intuitive de l'application

Les fonctionnalités de détection Tailscale, de refonte de la page de téléchargement et du mode CLI n'ont pas de cas de test automatisés. Elles ont été validées manuellement lors des tests End-to-End ajustés pour intégrer ces nouveautés. L'extension de la couverture des tests automatisés sur ces fonctionnalités est une piste d'amélioration identifiée pour une prochaine version.

Moyens de mise en œuvre :

Les tests unitaires et d'intégration sont automatisés avec pytest, en continuité du projet préTPI. Les tests End-to-End sont réalisés manuellement avec le même matériel qu'au préTPI (laptop Lenovo Yoga sous Fedora 43 (Linux) comme serveur, smartphone Android Realme Neo7 comme client).

Couverture :

Les tests ne sont pas exhaustifs. Ils couvrent les fonctionnalités principales et les points de sécurité. Les données de test reprennent celles du préTPI auxquelles s'ajoutent des tests spécifiques pour les fonctionnalités de CLI et de logging.

Environnement d'exécution des tests

Les tests automatisés sont exécutés avec pytest depuis le répertoire racine du projet. La configuration est définie dans `pytest.ini`, qui précise le répertoire `src/tests` comme chemin de recherche. La couverture de code est mesurée via le plugin `pytest-cov`.

La commande d'exécution complète est :

```
pytest --cov=src --cov-report=html
```

Résultats obtenus

L'ensemble des 9 cas de test définis passent avec succès. Le taux de couverture global est de 36% :

File ▲	statements	missing	excluded	coverage
src / cli.py	89	89	0	0%
src / controllers / app_controller.py	176	125	0	29%
src / controllers / hotspot_controller.py	52	33	0	37%
src / controllers / network_controller.py	50	41	0	18%
src / controllers / server_controller.py	111	90	0	19%
src / main.py	32	32	0	0%
src / models / model.py	125	54	0	57%
src / tests / test_controller.py	59	2	0	97%
src / tests / test_model.py	20	0	0	100%
src / tests / test_v2.py	35	0	0	100%
src / views / download_view.py	54	10	0	81%
src / views / view.py	155	133	0	14%
Total	958	609	0	36%
<i>coverage.py v7.13.5, created at 2026-05-18 13:31 +0200</i>				

Figure 3: Résultat de la couverture de tests

Le faible taux global s'explique par la difficulté de tester des composants liés à l'interface graphique Flet, dont les méthodes ne peuvent pas être testés automatiquement sans leur fenêtre flutter. Ces composants sont donc validés par les tests End-to-End manuels décrits plus haut.

2.4 Risques techniques

	Risque	Probabilité	Impact	Criticité	Solution
1	La création d'un point d'accès Wi-Fi ad-hoc avec python ne semble pas très courant, la fonctionnalité pourrait être fragile et rendre l'application instable	3	3	9	Explorer en détails les dépendances de la solution et essayer d'implémenter la fonctionnalité très tôt dans le projet pour rapidement détecter les problèmes.
2	Régression après l'ajout de fonctionnalités avancées	1	4	4	Tester l'application après chaque release afin de s'assurer au fur et à mesure que rien ne régresse.
3	La refonte de l'interface web pourrait introduire des régressions sur la logique de téléchargement de fichiers et de répertoires.	1	4	4	Maintenir les tests d'intégration existants et les exécuter systématiquement après chaque modification du frontend.
4	L'application est trop dure à utiliser pour un utilisateur final peu expérimenté	3	4	12	Refaire des maquettes et apporter des changements de design de l'application et de la page de téléchargement pour que l'expérience soit ergonomique
5	L'utilisation des fonctions actuels pour le CLI pourrait ne pas fonctionner car elles dépendent de l'interface graphique	4	4	16	Apporter des modifications aux fonctions actuels du programme afin de les rendre compatible avec l'utilisation par lignes de commandes

2.5 Planification

La planification définitive du projet reprend la structure en quatre sprints définie lors de l'analyse initiale. Le projet s'étend sur 90 heures réparties entre le 23 avril et le 20 mai 2026. Le diagramme de Gantt initial (Figure 1, section 1.5) a servi de référence, mais plusieurs ajustements ont été nécessaires en cours de réalisation.

Comparaison avec la planification initiale

Suite au blocage matériel du Sprint 1 (risque 1, de criticité 9), la User Story portant sur la création d'un hotspot pour permettre la connexion ad hoc, a été réévaluée à 5/5 sur la grille ci-dessus et la story a été reportée au Sprint 2. Cette fonctionnalité a finalement dû être repoussée deux fois (finie lors du Sprint 3). Le développement du point d'accès Wi-Fi ad hoc s'est heurté à deux contraintes matérielles : l'ordinateur fourni par le CPNV ne disposait d'aucune carte Wi-Fi, et le premier adaptateur USB que je me suis procuré ne supportait pas la fonctionnalité de création de réseau hébergé. Un adaptateur compatible a été trouvé le 06 mai avec l'aide de M. Wulliamoz et a permis de terminer le développement lors du deuxième Sprint, ainsi que de régler les soucis de packaging pour le Sprint 3. Cette fonctionnalité a donc été finalisée avec environ deux semaine de retard par rapport au planning initial.

Une autre source de retard a affecté plusieurs sprints : le comportement de l'application packagée (exécutable .exe généré par flet build windows) diverge de celui de l'application en environnement de développement. Ce comportement s'explique car la version packagée contient toutes les dépendances nécessaire pour tourner sur tous les environnements et affiche donc une fenêtre bloquant thread CLI pour le mode console et doit créé des fichiers, actions nécessitant des droits administrateurs spécifiques. Plusieurs régressions et problèmes spécifiques à l'exécutable (chemins de fichiers, CWD au lancement depuis le menu contextuel, attachement de console pour le mode CLI) ont nécessité des investigations non planifiées.

En réponse à ces aléas, la stratégie adoptée a été de reporter les stories non finalisées vers le sprint suivant, après analyse de faisabilité. Ainsi, la connectivité avancée et le mode CLI ont été terminés lors du Sprint 3, réduisant légèrement le temps consacré à l'expérience utilisateur (refonte de la page de téléchargement), qui constituait la dernière fonctionnalité à implémenter.

Des captures d'écrans de la section 'roadmap' de la page Github Projects de ce projet sont présents en annexe et peuvent être comparés au kanban du planning initial.

Maquettes de l'interface

Plusieurs maquettes ont été réalisées pour ce TPI, correspondant aux évolutions de l'interface par rapport au préTPI :

Maquette de l'application :

- Bouton démarrage/arrêt du serveur toujours en évidence
- ExpansionTile pour chaque code QR (cachés par défaut), permettant de ne pas inonder l'interface avec des code QR
- Ajout du paramétrage de la configuration du hotspot Wi-Fi ad hoc (SSID et mot de passe) reprenant la logique d'implémentation de la protection par mot de passe (via un ft.chip qui s'étend)

Maquette de la page de téléchargement : reflète les fonctionnalités à ajouter telles que décrites dans le cahier des charges (interface responsive, icônes par type de fichier, affichage de la taille des fichiers, bulk-downloads, refonte de l'interface de connexion). Les maquettes ont été réalisées avec Penpot, un outil de design open-source similaire à Figma déjà utilisé lors du préTPI.

La réalisation des maquettes en amont de chaque fonctionnalité a permis d'accélérer l'implémentation . Avant de créer une maquette, les widgets Flet nécessaires étaient identifiés dans l'application de galerie des composants, ce qui a permis d'assurer de leur présence. Puis ont été représentés plutôt fidèlement grâce à une librairie Penpot permettant d'utiliser les composants Material Design aussi utilisés par Flet.



Figure 4: Interface de l'application (configuration du partage)

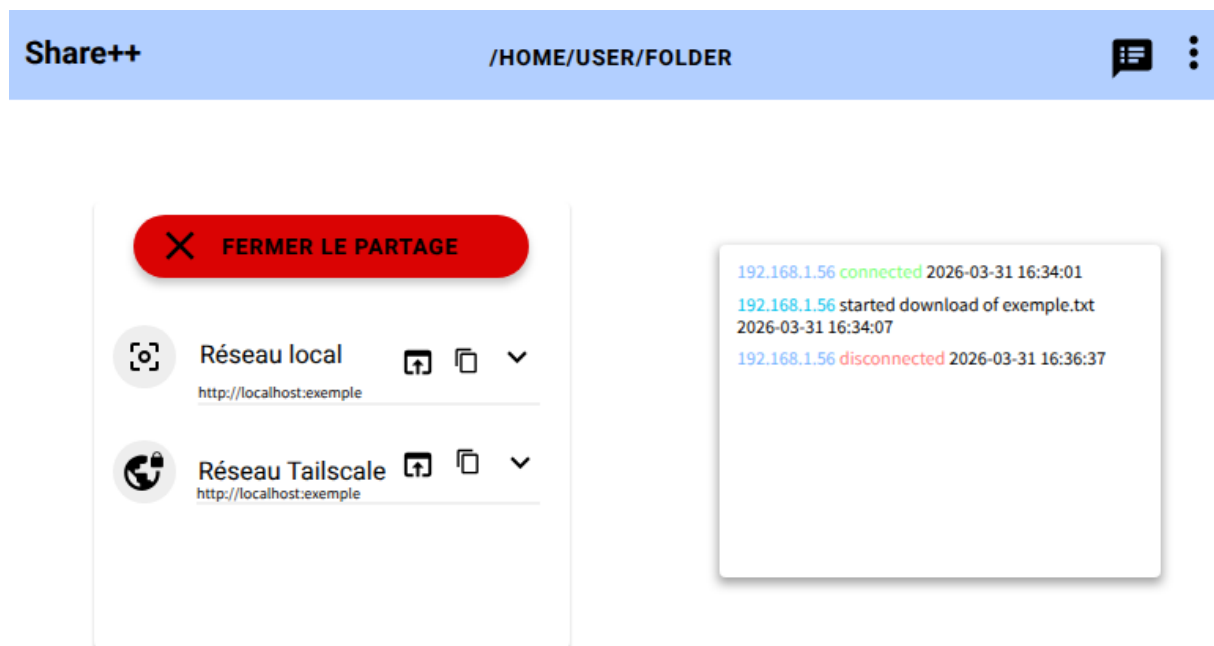


Figure 5: Interface de l'application (serveur ouvert sans hotspot)

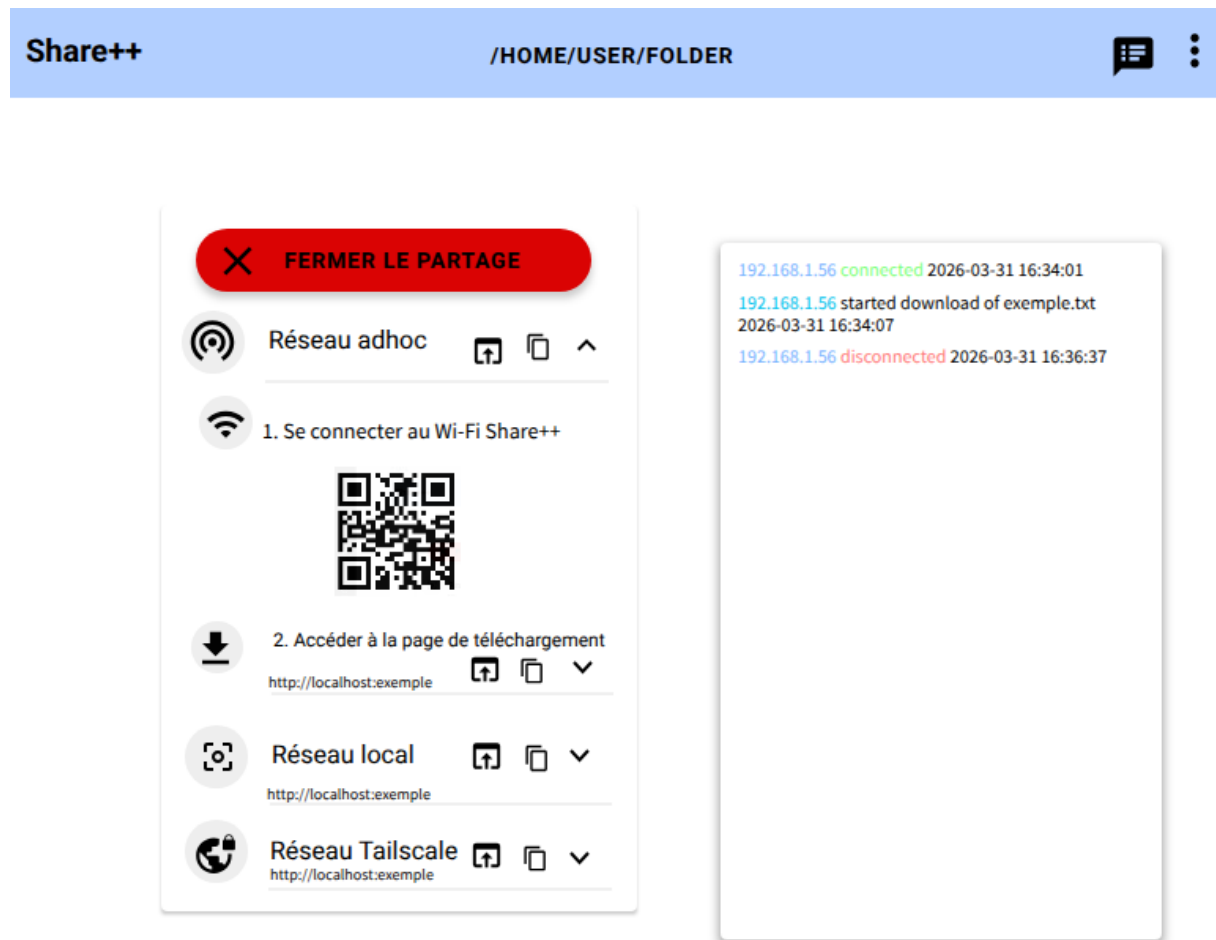


Figure 6: Interface de l'application (serveur ouvert avec hotspot)



Figure 7: Interface de l'historique de téléchargement

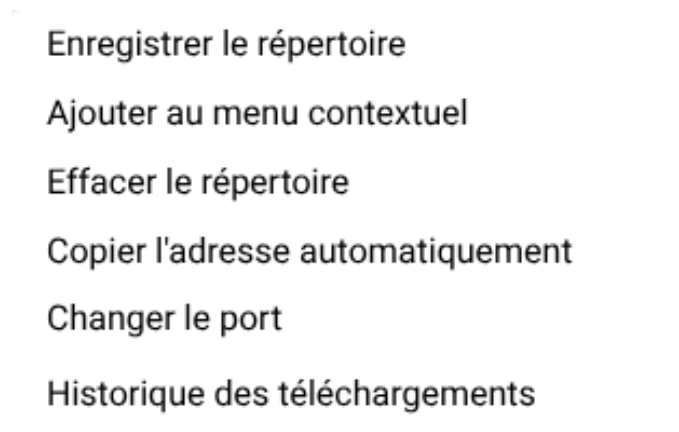


Figure 8: Interface des paramètres

2.6 Dossier de conception

Choix matériels et logiciels

Les choix technologiques sont détaillés dans le tableau de la section 1.2. L'environnement de développement principal reste le laptop personnel Lenovo Yoga sous Fedora 43 (Linux). Les tests et la validation finale sont effectués sur le poste Windows 11 mis à disposition par le CPNV, avec un smartphone Android realme Neo7 comme client de téléchargement.

L'exécutable produit par build de flet-cli est un binaire Windows autonome embarquant le runtime Flutter et le runtime Python. Contrairement à une application Electron qui embarque Chromium, ou à une application web progressive, l'interface graphique de Share++ est rendue nativement par le moteur Flutter (Skia/Impeller), sans navigateur ni WebView. Il serait donc inexact de qualifier cette application d'application web ou d'application hybride au sens courant du terme : il s'agit d'une application de bureau compilée dont le code UI est écrit en Python via l'abstraction Flet. Cette distinction a eu des implications concrètes sur le build : le processus d'initialisation Flutter démarre avant Python, ce qui a rendu impossible l'intégration du mode CLI dans le même exécutable (voir section 3.3.5).

Architecture de l'application

Le projet conserve l'architecture **MVC** (Model, View, Controller) établie lors du préTPI. Cette séparation garantit que chaque composant a une responsabilité unique, cela facilite la reprise du projet plus tard et la lisibilité du code.

Le découpage est le suivant :

Model (models/model.py) : gestion de la persistance des paramètres utilisateur (settings.json), du système de journalisation persistante (sharepp.log), et de toutes les opérations sur les données (hachage de mot de passe, toggles de configuration).

View (views/view.py, views/download_view.py) : interface de l'application graphique Flet et génération de la page HTML de téléchargement servie par aiohttp.

Controller : décomposé en quatre modules distincts pour éviter la monolithisation du contrôleur principal :

app_controller.py : orchestre les interactions entre l'UI Flet, le modèle et les contrôleurs spécialisés.

server_controller.py : logique du serveur aiohttp partagée entre le mode GUI et le mode CLI.

network_controller.py : détection des interfaces réseau disponibles (LAN, Tailscale, hotspot).

hotspot_controller.py : création et gestion du point d'accès Wi-Fi via netsh.

Diagramme de classe

Le diagramme de classe représentant l'architecture de l'application est disponible en annexe. Ce diagramme a été réalisé avec l'outil PlantUML. Outil très utilisé pour la création de diagrammes de classes en lignes de texte.

Modèle de persistance

Les données persistantes de l'application sont stockées dans deux fichiers situés dans %APPDATA%\SharePlusPlus\ :

settings.json : persistance des paramètres utilisateurs:

```
json{
  "selected_path": "/path/to/...",
  "port": 8080,
  "is_protected": false,
  "password": "",
  "logs_visible": true,
  "qr_enabled": true,
  "remember_path": true,
  "copy_to_clipboard": false,
  "context_menu_enabled": false,
  "hotspot_enabled": false,
  "hotspot_ssid": "SharePlusPlus",
  "hotspot_password": "sharepp1"
}
```

sharepp.log : journal persistant des événements, avec rotation automatique à 1 Mo (3 fichiers conservés) :

```
2026-04-27 12:06:43 [INFO]   Serveur ouvert sur le port 8080
2026-04-27 12:06:54 [INFO]   Connexion de 10.227.140.6 [PC (Linux)]
2026-04-27 12:06:54 [WARNING] 10.227.140.6 [PC (Linux)] télécharge fichier.pdf
2026-04-27 12:07:01 [ERROR]   Serveur fermé
```

3 Réalisation

3.1 Environnement

J'ai utilisé le même environnement pour ce travail que pour mon projet préTPI, c'est à dire que j'ai utilisé un laptop Linux personnel pour le développement, puis j'ai testé l'application depuis l'ordinateur sous Windows 11 mis à notre disposition par le CPNV. Tester l'application sur un poste Windows est important comme je développe sur un environnement techniquement non supporté par mon application (application windows développé sur un poste Linux). Il est donc possible de valider ainsi les fonctionnalités implémentées.

J'ai repris le dépôt Github du projet préTPI (<https://github.com/NathanFilipowitz/Sharepp>) et ai continué de l'utiliser pour ce projet, après avoir archivé l'état du dépôt dans une branche 'préTPI' dédiée. J'ai également créé des répertoires afin de ne pas garder tous les documents dans un seul répertoire /docs comme lors de mon dernier projet.

3.1.1 Développement sur laptop :

J'ai principalement utilisé mon laptop personnel pour réaliser ce travail. J'ai conservé la logique de séparation de code de mon projet préTPI. Les fichiers de code suivent la logique MVC (Model-View-Contrôleur) traduite ainsi dans mon projet :

- /src/controllers, code dédié à la logique métier de l'application. Les fonctionnalités, l'orchestration
- /src/models, code dédié à la persistance des données
- /src/views, code dédié à l'interface graphique de l'application et de la page de téléchargement

ainsi qu'un main.py servant de point d'entrée de l'application flet, et détecte désormais le niveau de privilège afin de pouvoir s'élever afin que la fonctionnalité de création d'access point fonctionne

Développement sur ordinateur CPNV :

J'ai eu besoin de migrer le projet sur l'ordinateur fixe du CPNV afin de pouvoir intégrer la fonctionnalité spécifique windows du bouton contextuel de l'explorateur de fichiers. Pour ceci, j'ai cloné le projet depuis mon dépôt github afin de pouvoir travailler sur l'environnement et pouvoir tester l'application directement. J'ai aussi utilisé cet environnement pour créer l'installateur InnoSetup pour le build final windows de l'application.

3.2 Structure du projet

La structure est également représentée de manière plus exacte dans le README du projet. L'architecture choisie est l'architecture MVC (Model, View, Controller) me permettant de séparer la logique de mon code.

```

Sharepp/
├── .github/
│   └── workflows/
│       └── build-release.yml        # Pipeline CI/CD automatisé
├── src/
│   ├── assets/
│   │   └── icons/
│   │       └── icon_windows.ico    # Icône utilisée par Inno Setup
│   ├── controllers/
│   │   ├── app_controller.py       # Orchestration GUI + logique métier
│   │   ├── hotspot_controller.py   # Création point d'accès Wi-Fi (netsh)
│   │   ├── network_controller.py   # Détection réseau (LAN + Tailscale)
│   │   └── server_controller.py     # Logique serveur aiohttp
│   ├── models/
│   │   └── model.py                # Persistance settings.json et logging
│   ├── tests/
│   │   ├── test_controller.py      # Tests d'intégration serveur aiohttp
│   │   ├── test_model.py           # Tests unitaires Model
│   │   └── test_v2.py              # TU-04 logging, TU-06 hotspot
│   ├── views/
│   │   ├── static/
│   │   │   ├── download.css        # Styles des pages HTML
│   │   │   └── download.js         # Logique du téléchargement multiple
│   │   ├── download_view.py        # Génération HTML
│   │   └── view.py                 # Interface graphique Flet
│   ├── cli.py                      # Application headless
│   └── main.py                     # Point d'entrée de l'application GUI
├── .gitignore
├── LICENSE
├── README.md
├── pyproject.toml                  # Config Flet, métadonnées projet
├── requirements.txt                # Dépendances Python
└── setup.iss                       # Script Inno Setup

```

3.3 Fonctionnalités techniques importantes

3.3.1 Workflow Build & Release automatisé

Un workflow GitHub Actions (.github/workflows/build-release.yml) automatise entièrement le build windows de l'application, ainsi que le packaging et la publication de l'installateur Windows sous forme de release versionnée sur le dépôt Github. Il se déclenche à chaque push s'accompagnant d'un tag de version -v*.

```
name: Build & Release Windows Installer
```

```
on:  
  push:  
    tags:  
      - 'v*'
```

Figure 9: Déclenchement du workflow par tag de version

Etapes du workflow

1. Récupération du code du dépôt (actions/checkout@v4) et configuration de Python 3.12.
2. Extraction de la version depuis le tag Git via PowerShell et injection dans les variables d'environnement GitHub Actions.
3. Installation des dépendances Python depuis requirements.txt.
4. Installation du SDK Flutter (v3.41.4) via subosito/flutter-action@v2.
5. Build de l'exécutable Windows via flet build windows, avec la variable PYTHONUTF8=1 pour éviter les erreurs d'encodage. Le fichier requirements.txt est copié dans src/ pour que Flet trouve les dépendances.
6. Renommage de l'exécutable généré : src.exe > sharepp.exe (flet build windows nomme l'exécutable d'après le dossier source nommé 'src').
7. Installation d'Inno Setup via Chocolatey.
8. Mise à jour du numéro de version dans setup.iss.
9. Génération de l'installateur .exe via ISCC.exe (Inno Setup Command-Line Compiler).
10. Build de l'exécutable CLI sharepp-cli.exe via PyInstaller (--onefile --console), avec inclusion explicite des répertoires views, controllers et models.
11. Création de la release GitHub et upload des deux artefacts : SharePlusPlus_Setup_{version}.exe et sharepp-cli.exe, via softprops/action-gh-release@v2.

Ce pipeline garantit la reproductibilité des releases, leur versionnement et permet aux utilisateurs de pouvoir facilement et rapidement accéder à une version à jour et facilement installable de l'application.

3.3.2 Logging

- Utilisation de la librairie « logging » de la librairie standard de python.
- Écriture dans un fichier sharepp.log avec rotation automatique pour éviter les fichiers logs trop volumineux.
- Ajout d'une nouvelle donnée dans les logs : Le type d'appareil utilisé pour accéder au partage (récupéré grâce au User-Agent parsing).
- Intégration dans le système précédent de logs sans trop de réécriture

L'application disposait déjà d'un affichage des logs en temps réel dans son interface dès sa version 1.0 (préTPI), mais ces informations étaient perdues à chaque fermeture. Pour répondre à l'objectif de traçabilité du cahier des charges, un système de journalisation persistant a été ajouté au modèle, permettant de conserver un historique des logs précédemment affichés qu'en temps réel. Ceci permet à l'utilisateur de pouvoir consulter les logs a posteriori, sans avoir à ouvrir l'application. L'utilisateur aura également droit à d'avantage d'informations sur le type d'appareil (« [Mobile[Android]] », « [PC[Windows]] »...) qui s'est connecté à son serveur.

La librairie logging de python repose sur trois concepts : le **Logger**, qui reçoit les messages et filtre par niveau de sévérité ; le **Handler**, qui dirige les messages vers leur destination (ici un fichier `sharepp.log`), et le **Formatter**, qui définit la mise en forme de chaque entrée.

```
# Persistent log file path
self.log_path = self.config_dir / "sharepp.log"
self._setup_file_logger()

def _setup_file_logger(self):
    # Identify log file (create if not exists)
    self.file_logger = logging.getLogger("sharepp")
    # Accept logs of all severity levels (DEBUG == lowest priority)
    self.file_logger.setLevel(logging.DEBUG)

    if not self.file_logger.handlers:
        handler = RotatingFileHandler(
            self.log_path, maxBytes=1_000_000, backupCount=3, encoding="utf-8"
        )
        formatter = logging.Formatter(
            "%(asctime)s [%(levelname)s] %(message)s",
            datefmt="%Y-%m-%d %H:%M:%S"
        )
        handler.setFormatter(formatter)
        self.file_logger.addHandler(handler)
```

Figure 10: Capture d'écran de la logique d'initialisation du logger

```
def log_to_file(self, message: str, level: str):
    level_map = {
        "success": logging.INFO,
        "info": logging.INFO,
        "warning": logging.WARNING,
        "error": logging.ERROR,
    }
    self.file_logger.log(level_map.get(level, logging.INFO), message)
```

Figure 11: Capture d'écran du dictionnaire des niveaux de logs

Emplacement du fichier de log

Le fichier logs est enregistré dans le répertoire Appdata de Windows, aux côtés de settings.json, ce répertoire est le répertoire conseillé pour les documents dont il faut le droit d'accès en écriture.

Retrouver le fichier :

%APPDATA% -> Roaming\SharePlusPlus\sharepp.log

Format des logs

2026-04-27 12:06:43 [INFO] Serveur ouvert sur le port 8080

2026-04-27 12:06:54 [INFO] Connexion de 10.227.140.6 [PC (Linux)]

2026-04-27 12:06:54 [WARNING] 10.227.140.6 [PC (Linux)] téléchargement favicon.ico

2026-04-27 12:07:01 [ERROR] Serveur fermé

Rotation automatique

Pour ne pas réécrire continuellement sur le même fichier log, un RotatingFileHandler a été utilisé, servant à créer un nouveau fichier log après que celui-ci ait dépassé 1 Mo, et ne conserve que trois versions de fichiers logs. Comme toutes les logs affichés sur l'application sont enregistrés automatiquement, cela garantit que l'espace disque autorisé pour les logs soit limité à 4 Mo.

Intégration avec le contrôleur

La méthode Controller.log(message, niveau) appelle désormais model.log_to_file() pour la persistance du log, et continue de l'afficher dans l'interface Flet en même temps.

Identification des appareils (User-Agent Parsing)

Chaque entrée de log issue d'une connexion entrante, inclut une identification de l'appareil client déduite par le header HTTP User-Agent. La méthode parse_user_agent() du contrôleur récupère le type d'appareil (Mobile, Tablette, PC) et le système d'exploitation (Android, iOS, Windows, Linux, macOS) de chaque requête de téléchargement entrante (pas pour les logs informatifs de type Password protection enabled).

Exemples de sorties :

Mobile (Android)

PC (Windows)

Appareil inconnu


```
# Identify device type and OS from HTTP User-Agent to render something readable for the user
def parse_user_agent(ua):
    if not ua:
        return "Appareil inconnu"

    if "Mobile" in ua or "Android" in ua:
        device = "Mobile"
    elif "Tablet" in ua or "iPad" in ua:
        device = "Tablette"
    else:
        device = "PC"

    if "Android" in ua:
        os_name = "Android"
    elif "iPhone" in ua or "iPad" in ua:
        os_name = "iOS"
    elif "Windows" in ua:
        os_name = "Windows"
    elif "Linux" in ua:
        os_name = "Linux"
    elif "Mac" in ua:
        os_name = "macOS"
    else:
        os_name = "OS inconnu"

    return f"{device} ({os_name})"
```

Figure 12: Capture d'écran du parsing de l'appareil utilisé par le client

Page d'historique des téléchargements

Le fichier sharepp.log sert également à récupérer les derniers téléchargements directement depuis l'application. Un bouton ajouté au PopMenuButton « Voir l'historique des téléchargements » affiche une fenêtre présentant les derniers noms de fichiers partagés, incluant leur métadonnées et la possibilité d'ouvrir le fichier dans l'explorateur windows s'ils sont toujours présent et au même emplacement.

Parsing du fichier log

La méthode get_download_history() ajouté au modèle lit le fichier sharepp.log et en extrait les entrées de téléchargement via l'expression régulière suivante :

`(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}) \[WARNING\] ([\d\.]+) \[(.+?)\] télécharge '(.)'`

Cette expression régulière cible les entrées de niveau [WARNING] correspondant à un téléchargement, et capture quatre groupes : l'horodatage, l'adresse IP du client, le type d'appareil et le nom du fichier. Les résultats sont retournés dans l'ordre inverse afin d'afficher le téléchargement le plus récent en premier.

Interface de l'historique des téléchargements

Chaque entrée de l'historique est affichée sous forme d'une carte avec le nom du fichier en gras, l'horodatage, l'adresse IP et le type d'appareil. Un bouton d'ouverture dans l'Explorateur Windows est disponible pour chaque entrée : il est actif uniquement si le dossier partagé est encore sélectionné dans l'application et si le fichier existe toujours à son emplacement d'origine. Si ces conditions ne sont pas remplies, le bouton est désactivé. L'ouverture se fait via `subprocess.run(["explorer", "/select,", filepath])`, ce qui ouvre l'Explorateur avec le fichier sélectionné.

3.3.3 Point d'accès Wi-Fi

Le cas d'usage principal de Share++ consiste à partager des fichiers entre un ordinateur de bureau et un smartphone. Cependant, il est commun que le smartphone et l'ordinateur ne partagent pas le même sous-réseau, si le smartphone est connecté en 5g par exemple. Dans ce contexte, le client mobile n'est pas en mesure d'accéder à la page de téléchargement générée par le serveur aiohttp.

La solution retenue est de permettre à l'hôte de créer lui-même un réseau Wi-Fi ad hoc temporaire. Le smartphone se connecte à ce réseau, rejoignant ainsi le sous-réseau de l'hôte et rendant la page accessible.

Implémentation

La logique de création, démarrage et arrêt du point d'accès est inscrite dans le fichier `src/controllers/hotspot_controller.py`, afin de respecter l'architecture MVC du projet.

Fonctions ajoutées

- `get_hotspot_status()` : lance la commande ``netsh wlan show hostednetwork`` et détecte si le point d'accès est actif, en traitant la sortie de la commande en français et en anglais (bilingue, car le retour de la commande sur la version francophone de windows est en français).
- `setup_and_start(ssid, password)` : valide le mot de passe (minimum 8 caractères, exigence de tout mot de passe Wi-Fi WPA2), exécute la commande de configuration puis démarre le réseau.
- `stop_hotspot()` : arrête le point d'accès.
- `execute_command(command)` : wrapper autour de `subprocess.run` avec le flag `CREATE_NO_WINDOW (0x08000000)` pour éviter les flashes de terminal Windows. Implémenté avec l'aide de Gemini (voir journal de travail).
- `ensure_admin()` : vérifie les droits administrateur et relance l'application avec élévation si nécessaire.

```
def setup_and_start(ssid, password):  
    if len(password) < 8: return False, "Le mot de passe doit faire 8 caractères."  
    if platform.system() == "Windows":  
        # Configuration  
        cmd_config = f'netsh wlan set hostednetwork mode=allow ssid="{ssid}" key="{password}"'  
        ok, msg = execute_command(cmd_config)  
        if not ok: return False, f"Erreur de configuration: {msg}"  
        # Access Point Creation  
        return execute_command("netsh wlan start hostednetwork")  
  
    return False, "OS non supporté"
```

Figure 13: Capture d'écran de la fonction de démarrage du point d'accès Wi-Fi

```
def get_hotspot_status():  
    # Check if hotspot is active  
    if platform.system() == "Windows":  
        ok, output = execute_command("netsh wlan show hostednetwork")  
        # check in results for french and english  
        is_running = "Status : Started" in output or "État : Démarré" in output  
        return {"running": is_running}  
    return {"running": False}
```

Figure 14: Capture d'écran de la fonction vérification de l'état du hotspot

```
def stop_hotspot():  
    if platform.system() == "Windows":  
        return execute_command("netsh wlan stop hostednetwork")  
    return False, "OS non supporté"
```

Figure 15: Capture d'écran de la fonction d'arrêt du hotspot

Intégration dans l'interface

La hotspot card indépendante présente dans une version antérieure a été supprimée pour une intégration dans la card principale de configuration du partage. Deux contrôles ont été ajoutés :

Un bouton "Créer un hotspot Wi-Fi" permet d'activer la fonctionnalité de manière persistante dans settings.json. Ce bouton est affiché sur tous les systèmes d'exploitation mais est désactivé sur les systèmes non-Windows.

Un bouton "Configurer le réseau Wi-Fi" ouvre un AlertDialog contenant les champs SSID et mot de passe. Conformément à l'architecture MVC, la création et l'affichage de ce dialog sont entièrement gérés dans view.py le contrôleur se limite à valider et persister les données via save_hotspot_credentials().

Flux de l'expérience utilisateur

Lorsque l'utilisateur active le hotspot depuis l'interface :

1. Share++ crée le réseau avec le SSID et le mot de passe configurés.
2. Une bannière d'instruction apparaît au-dessus des tuiles de connexion, invitant l'utilisateur à se connecter d'abord au réseau Wi-Fi avant de scanner l'URL.
3. Un QR code Wi-Fi est disponible dans une ExpansionTile dédiée, encodant la chaîne `WIFI:T:WPA;S:{ssid};P:{pwd};H;;` qui permet de se connecter automatiquement en le scannant depuis un smartphone.
4. L'adresse IPv4 de la machine servant le serveur de fichier : `http://192.168.137.1:{port}` apparaît comme option de connexion supplémentaire avec un QR code de connexion aux côtés des adresses de l'ip locale et de l'ip Tailscale.
5. Une fois connecté au hotspot, l'utilisateur peut scanner le QR code issu du hotspot adhoc pour accéder aux fichiers. A l'arrêt du serveur de fichier, le réseau adhoc se ferme automatiquement.

Problèmes rencontrés

L'utilisation de netsh (interpréteur de commandes réseau Windows) pour la création de l'access point oblige à tester la fonctionnalité sur une machine Windows. Le problème étant que l'ordinateur de test mis à ma disposition pour ce travail n'avait pas de carte Wi-Fi et ne pouvait donc pas afficher les commandes netsh recherchées. J'ai donc dû me procurer une clé USB Wi-Fi afin de pouvoir tester et corriger l'implémentation.

Même après avoir obtenu la clé Wi-Fi TP-Link et installé les drivers, je me suis rendu compte que la clé ne supportait pas la fonctionnalité 'réseau hébergé' (netsh wlan show drivers -> Réseau hébergé pris en charge : non), prérequis nécessaire à la commande netsh wlan hostednetwork.

Le Mercredi 06 mai, M. Wulliamoz et moi avons testé plusieurs adaptateurs USB ainsi qu'un laptop windows, jusqu'à ce que nous ayons trouvé un adaptateur USB qui supporte la fonctionnalité me permettant alors de poursuivre l'implémentation avec un peu de retard.

3.3.4 Détection et intégration Tailscale

Contexte et objectif

Share++ est conçu pour fonctionner sur un réseau local. Cependant, il est fréquent qu'un utilisateur souhaite partager des fichiers avec un appareil situé hors de son réseau (comme avec un smartphone en 5g par exemple), sans pour autant avoir configuré manuellement une redirection de port sur son routeur. Tailscale est un service de réseau maillé sécurisé basé sur WireGuard qui résout exactement ce problème : chaque appareil du sous-réseau Tailscale appelé "Tailnet" reçoit une adresse IP fixe de plage 100.x.x.x et peut communiquer avec les autres machines du Tailnet, pourvu qu'ils aient un accès à internet.

L'objectif est de détecter automatiquement la présence du service Tailscale sur la machine hôte au moment du démarrage du serveur, et de proposer un lien de téléchargement supplémentaire via cette interface, sans aucune configuration manuelle de la part de l'utilisateur.

Architecture `network_controller.py`

La logique de détection réseau a été écrite dans un fichier dédié : `src/controllers/network_controller.py`, conformément à l'architecture MVC du projet. Ce contrôleur expose trois fonctions publiques :

`get_local_ip()` : détecte l'adresse IP du réseau local (précédemment directement présent dans `app_controller.py`).

`get_tailscale_ip(port)` : détecte l'adresse IP Tailscale et valide son accessibilité.

`get_hotspot_ip()` : détecte l'adresse IP de l'interface du point d'accès Wi-Fi ad hoc (voir section 3.3.3).

Détection de l'adresse IP Tailscale

Pour détecter la présence de Tailscale sur la machine de l'utilisateur, on lance une commande tailscale CLI permettant d'afficher l'ip Tailscale de la machine.

La détection se déroule en deux étapes. D'abord, `tailscale status` est exécuté via `subprocess.run()` avec un timeout de 3 secondes. Son code de retour détermine si le service est actif. Si le service est inactif, aucun changement dans l'application ne sera observé.

Si le service est actif, `tailscale ip --4` est exécuté et récupère l'adresse IPv4 de Tailscale. L'adresse est aussi validée par une expression régulière garantissant qu'elle appartient bien à la plage Tailscale.

Validation de l'adresse IP Tailscale

Pour s'assurer que la connexion à la page de téléchargement fonctionne correctement via l'IP Tailscale, une méthode privée `_is_reachable()` a été implémentée, lançant une tentative de connexion via la librairie `socket` afin de pouvoir proposer la fonctionnalité à l'utilisateur, que si elle fonctionne vraiment.

```
def _is_reachable(ip, port, timeout: float = 3.0):
    try:
        with socket.create_connection((ip, port), timeout=timeout):
            return True
    except OSError:
        return False

def _detect_tailscale_ip():
    # Is tailscale active: "tailscale status" output
    try:
        status = subprocess.run(
            ["tailscale", "status"],
            capture_output=True, text=True, timeout=3
        )
        if status.returncode != 0:
            return None

        # get tailscale ip for this machine in IPv4
        result = subprocess.run(
            ["tailscale", "ip", "--4"],
            capture_output=True, text=True, timeout=3
        )
        ip = result.stdout.strip()
        if ip and re.match(r"^100\.(6[4-9]|[7-9]\d|1[0-1]\d|12[0-7])\.\d{1,3}\.\d{1,3}$", ip):
            return ip

    except Exception as e:
        print(f"Error during Tailscale ip detection: {e}")

    return None
```

Figure 16: Capture d'écran de la détection de l'adresse IP Tailscale et de la validation de son accessibilité

Alternative envisagée

En connaissant la plage standard d'attribution d'adresses IPV4 de Tailscale, il aurait été possible de récupérer les adresses IPV4 de la machine via la librairie python standard 'socket'. Cela aurait eu l'avantage de fonctionner en l'absence de tailscale dans le PATH. Cette méthode n'a pas été retenue car une adresse pourrait commencer par 100. sans venir de Tailscale, rendant la détection moins fiable. La méthode CLI ayant fonctionné directement, cette alternative n'a pas été poursuivie.

Validation de l'accessibilité

La fonction `get_tailscale_ip(port)` valide l'adresse IPV4 Tailscale détectée via une tentative de connexion TCP (`socket.create_connection`) sur le port du serveur. Cela sert à valider le bon fonctionnement du lien de téléchargement hors-site avant de le proposer à l'utilisateur.

`get_local_ip()`

La fonction `get_local_ip()` détecte l'IP locale via une connexion UDP factice vers 8.8.8.8. L'implémentation reprend du code directement de la documentation officielle disponible dans la section sources de ce document.

Intégration dans le flux de démarrage du serveur

La détection est déclenchée automatiquement à chaque démarrage du serveur de fichier, dans la méthode `start_server()` du contrôleur. Elle s'exécute dans un thread séparé via `asyncio.to_thread()` afin de ne pas bloquer la boucle d'événements Flet pendant le scan réseau (`subprocess.run()` est bloquant) :

```
local_ip, tailscale_ip, hotspot_ip = await asyncio.to_thread(self._detect_all_ips)
await self._show_connection_info(local_ip, tailscale_ip, hotspot_ip)
```

Figure 17: Capture d'écran montrant la détection des interfaces réseau disponibles dans un thread séparé

La méthode `_show_connection_info()` génère ensuite les QR codes et met à jour l'interface selon les IPs disponibles :

Si seul le LAN est disponible : un seul ExpansionTile QR est affiché (comportement inchangé par rapport à la version préTPI).

Si Tailscale est détecté : deux ExpansionTiles indépendants sont affichés, un pour l'adresse LAN et un pour l'adresse Tailscale.

Aucune donnée liée à Tailscale n'est inscrite dans le fichier `settings.json`. La détection est donc entièrement en mémoire et se relance à chaque session afin de rendre l'expérience utilisateur plus fiable.

Interface utilisateur

L'affichage des QR codes a alors été refait pour épurer l'interface utilisateur. Les QR codes ne s'affichent plus en grand au démarrage du serveur mais sont masqués par défaut dans des composants `ft.ExpansionTile`. Chaque tile expose sur lui de manière toujours visible :

L'URL du partage sélectionnable.

Un bouton d'ouverture dans le navigateur.

Un bouton de copie dans le presse-papier.

Le bouton d'arrêt du serveur reste toujours visible au-dessus des ExpansionTiles.

3.3.5 Utilisation en ligne de commande (CLI)

Contexte et objectif

Share++ est conçu comme une application graphique Windows. Cependant, certains power users apprécieront d'avoir la possibilité d'ouvrir un serveur de fichier sécurisé en une ligne de commande directement dans leur terminal. L'objectif de cette fonctionnalité de permettre à l'utilisateur d'utiliser **l'outil** Share++ sans passer par son interface graphique, en préservant les fonctionnalités principales telle que le qr code, la protection par mot de passe, les logs.

Analyse des variantes

Deux approches principales ont été envisagées pour l'implémentation de cette fonctionnalité :

- Variante 1 Exécutable unique : main.py détecte la présence de l'argument --nogui au démarrage et délègue l'exécution à un module CLI dédié, sans jamais initialiser Flet.
- Variante 2 Exécutable séparé : un second exécutable sharepp-cli.exe est buildé indépendamment, sans aucune dépendance Flet, produisant un binaire plus léger.

Méthode retenue

La deuxième variante a été poursuivie après avoir rencontré un problème bloquant avec la première variante. En effet, il n'était simplement pas possible d'appeler sharepp.exe build sans qu'une fenêtre vide ne s'affiche, bloquant par la même occasion le thread du module CLI. Cette limitation est inhérente à l'architecture de Flet : le runtime Flutter démarre avant Python et initialise systématiquement une fenêtre graphique, indépendamment des arguments passés au script (voir section 2.5). Pour créer un exécutable séparé, une étape de build de l'exécutable via pyinstaller a été ajoutée au workflow build-release. La version CLI est alors proposée en dessous de la release de l'application GUI.

```
def parse_args():
    parser = argparse.ArgumentParser(
        prog="sharepp-cli",
        description="Share++ CLI, serveur de partage de fichiers en réseau local en lignes de commandes.",
    )
    # Positional argument : mandatory, doesn't use --
    parser.add_argument("path", help="Répertoire à partager")
    # Optional arguments
    parser.add_argument("--port", type=int, default=8080, help="Port HTTP (défaut : 8080)")
    parser.add_argument("--secure", action="store_true", help="Active la protection par mot de passe")

    return parser.parse_args()
```

Figure 18: Capture d'écran de la définition des arguments de l'application en ligne de commande

3.3.6 Refonte de la page de téléchargement

Contexte et objectif

La page de téléchargement servie par aiohttp affiche les fichiers servis par Share++ aux appareils client. Dans la version préTPI, cette page se limitait à une liste de fichiers avec un bouton de téléchargement par entrée, sans métadonnées, sans responsive, et avec une interface d'authentification reposant sur le mécanisme natif WWW-Authenticate: Basic, qui affichait un popup navigateur natif avec deux champs (utilisateur et mot de passe), la solution n'était pas idéale car Share++ n'intègre pas la notion d'utilisateurs.

L'objectif de ce sprint était de réécrire entièrement cette interface pour la rendre plus pratique à l'utilisation en intégrant les points suivants : adapter l'interface aux appareils mobiles (responsive design), entrées des fichiers plus informative (ajout des métadonnées (horodatage, taille du fichier) et de styles, créer une nouvelle page de connexion dédiée pour entrer le mot de passe du partage.

Architecture des fichiers

La logique de présentation est désormais répartie sur trois fichiers dans `src/views/` :

```
src/views/  
├── download_view.py    # génération HTML + traitement métadonnées  
├── static/  
│   ├── download.css    # styles  
│   └── download.js      # sélection multiple et bulk download
```

`server_controller.py` expose ces fichiers statiques via `app.router.add_static("/static", _STATIC_DIR)`. La route `/static/` est enregistrée avant `{filename}` pour éviter toute ambiguïté avec les fichiers partagés.

Page de téléchargement

La fonction `generate_html(files, shared_path)` reçoit désormais `shared_path` en plus de la liste de fichiers, afin de lire la taille et la date de modification réelles de chaque fichier via `os.path.getsize` et `os.path.getmtime`. Pour chaque fichier, la page affiche une icône colorée selon le type, le nom du fichier, sa taille en format lisible (o / Ko / Mo / Go) et sa date de dernière modification.

Les icônes SVG sont issues de Feather Icons (licence MIT). Elles sont définies dans une liste de dictionnaires `FILE_TYPES`, chaque entrée contenant la liste des extensions associées, une couleur hexadécimale et les paths SVG correspondants. Deux fallbacks existent : `ICON_FOLDER` pour les répertoires et `ICON_UNKNOWN` pour les extensions non reconnues.

Les fichiers cachés (nom commençant par `.`) reçoivent la classe CSS `hidden-file` : opacité réduite et italique, pour les signaler visuellement sans les masquer.

Sélection multiple (bulk download)

Une checkbox placée à gauche de chaque carte permet la sélection multiple. Un bouton de téléchargement groupé, masqué par défaut, apparaît dès qu'au moins une checkbox est cochée et indique le nombre de fichiers sélectionnés. Les fichiers se téléchargent alors l'un après l'autre avec un intervalle de 400 ms entre chaque fichier afin de prévenir des restrictions du navigateur. La logique de sélection et de téléchargement est entièrement gérée dans `download.js`.

Page d'authentification

Le mécanisme WWW-Authenticate: Basic a été remplacé par une page HTML servie sur la route `/password`. Cette page ne contient qu'un champ mot de passe, sans champ utilisateur. En cas d'erreur, un message s'affiche dans la même page sans rechargement complet. Le CSS de cette page est inclus dans `download.css` sous une section dédiée appliquée via `body.auth-page`, pour ne pas interférer avec les styles de la page liste.

Authentification par cookie signé

Le middleware `make_auth_middleware` a été mis à jour pour vérifier un cookie de session signé avec HMAC-SHA256 au lieu du header `Authorization: Basic`. Les routes `/password` et `/static/*` sont toujours autorisées afin que le CSS soit accessible pour l'authentification. Lorsque le mot de passe soumis est correct, un cookie `sharepp_session` est posé (`httponly`, `SameSite=Strict`), signé avec une clé aléatoire générée au démarrage (`os.urandom(32)`). Ce mécanisme invalide automatiquement toutes les sessions à chaque redémarrage d'un serveur de fichier Share++.

```
# return the aiohttp password authentication middleware using the custom HTML password form.
def make_auth_middleware(password_hash):
    @web.middleware
    async def auth_middleware(request, handler):
        if request.path == "/password" or request.path.startswith("/static/"):
            return await handler(request)

        # Check session cookie
        cookie = request.cookies.get(_COOKIE_NAME, "")
        if cookie and _is_session_valid(cookie, password_hash):
            return await handler(request)

        # Redirect to password page if user is not authenticated
        raise web.HTTPFound("/password")

    return auth_middleware
```

Figure 19: Capture d'écran du middleware d'authentification par cookie

```
# random sequence used to sign the session cookie. A new one is generated on each server startup
_COOKIE_SECRET = os.urandom(32)
_COOKIE_NAME = "sharepp_session"

# return HMAC-signed token proving the user entered the correct password
def _make_session_token(password_hash):
    sig = hmac.new(_COOKIE_SECRET, password_hash.encode(), hashlib.sha256).hexdigest()
    return sig
```

Figure 20: Capture d'écran de la génération aléatoire du cookie

3.4 Liste des documents fournis

Les éléments suivants constituent la livraison finale du projet TPI Share++.

Document	Format	Description
Dossier de projet	PDF / ODT	Document principal comprenant l'analyse, la conception et la réalisation
Journal de travail	ODS	Rapports journaliers des activités, succès, échecs et aides externes
Journal de test	PDF / ODS	Plan de test et résultats d'exécution des cas de test

Maquette application	PNG	Maquette Penpot de l'interface graphique Flet (version TPI)
Maquette page de téléchargement	PNG	Maquette Penpot de la page web de téléchargement refaite
Schéma réseau	PNG / drawio	Schéma de l'architecture réseau client-serveur
Code source	GitHub	Dépôt public NathanFilipowitz/Sharepp
Installateur Windows	EXE	SharePlusPlus_Setup_2.0.0.exe, assistant d'installation GUI
Exécutable CLI	EXE	sharepp-cli.exe, version headless en ligne de commande

3.5 Résultat des tests

Les 10 cas de test automatisés ont été exécutés avec succès. Le taux de couverture global est de 36 %, ce qui s'explique par l'impossibilité de tester automatiquement les composants liés à l'interface graphique Flet sans son moteur Flutter. Ces composants sont validés par les tests End-to-End manuels. Les 3 cas manuels (TE-01, TE-02, TP-01) ont également été validés avec succès.

Environnement d'exécution des tests

Les tests automatisés sont exécutés avec pytest depuis le répertoire racine du projet. La configuration est définie dans pytest.ini, qui précise src/tests comme chemin de recherche. La couverture de code est mesurée grâce au plugin pytest-cov.

Commande d'exécution complète utilisée :

```
pytest --cov=src --cov-report=html
```

Détails de l'environnement utilisé :

- Smartphone Android Realme Neo7 (client de téléchargement pour les tests End-to-End)
- Ordinateur Windows fourni par le CPNV
- Laptop personnel fedora 43
- utilisateurs tiers pour TP-01.

ID	Description	Type	Donnée d'entrée	Résultat attendu	Résultat
TU-01	Persistence JSON	Unitaire	config_name="test_settings.json", tmp_path	Fichier créé, data["port"] == 8080	PASS
TU-02	Hashage password	Unitaire	password="reallyGoodPassword"	Hash SHA-256 du password stocké (pas de mot de passe en dur)	PASS

TU-03	Génération QR	Unitaire	"http://192.168.1.10:8080"	Résultat commence par "data:image/png;base64,"	PASS
TU-04	Logs persistents	Unitaire	Appels log_to_file() avec niveaux info, warning, error, success	Fichier .log contient [INFO], [WARNING], [ERROR] et le message "serveur démarré"	PASS
TU-05	Validation mot de passe hotspot positif	Unitaire / Sécurité	Mots de passe "12345678"	execute_command appelé au moins une fois, retour (True, "OK")	PASS
TU-06	Validation mot de passe hotspot négatif	Unitaire / Sécurité	"1234567"	Retour False sans aucun appel à execute_command	PASS
TI-01	Génération HTML	Intégration	Dossier simulé avec test.txt et subfolder/	HTTP 200, HTML contient "test.txt" et "subfolder"	PASS
TI-02	Blocage traversal	Intégration	Requête GET ../secret.txt	HTTP 403 ou 404	PASS
TI-03	Zip du répertoire	Intégration	Requête GET /subfolder	HTTP 200, Content-Type: application/zip (header fichier zip)	PASS
TE-01	Utilisation smartphone	End-to-End	Protection activée, réseau local, smartphone Android	Page d'authentification affichée, redirection après mot de passe correct, téléchargement fonctionnel, logs en temps réel	PASS

TE-02	hotspot ad hoc + partage complet (Windows)	End-to-End	PC Windows CPNV, adaptateur Wi-Fi compatible, smartphone Android	Connexion au réseau hotspot via QR Wi-Fi, accès à la page de téléchargement, fichier reçu	PASS
TP-01	Utilisation par utilisateur tiers	Utilisateur	Utilisateur novice, fichier test.txt dans Documents	Navigation sans blocage, actions intuitives, aucune frustration observée	PASS

Les tests valident les fonctionnalités implémentées lors du préTPI et lors du deux premiers sprints. Les fonctionnalités des sprints suivants (refonte de la page web, CLI, détection Tailscale) ont été validées manuellement en raison des contraintes matérielles et de temps décrites en section 2.4 qui n'ont pas permis d'étendre la suite de tests dans les délais impartis. L'extension de la couverture automatisée constitue une piste d'amélioration identifiée pour une version future.

4 Conclusions

4.1 Objectifs atteints

Les cinq axes d'amélioration définis dans le cahier des charges ont été implémentés :

- **Traçabilité et audit** : le système de journalisation persistante (`sharepp.log`) avec rotation automatique et identification des appareils par User-Agent parsing est pleinement fonctionnel et testé.
- **Partage à distance** : la détection automatique de Tailscale est implémentée et intégrée au flux de l'utilisation de l'application. L'adresse Tailscale est proposée à l'utilisateur uniquement si le service est actif **et** l'adresse IP joignable.
- **Connectivité avancée** : le point d'accès Wi-Fi ad hoc via `netsh wlan hostednetwork` est fonctionnel sur une machine Windows compatible (disposant d'une carte Wi-Fi supportant l'hébergement réseau). La fonctionnalité est désactivée proprement sur les systèmes non supportés.
- **Automatisation CLI** : `sharepp-cli.exe` est un exécutable autonome généré via PyInstaller, distribué dans la même release GitHub que l'installateur GUI.
- **Expérience utilisateur** : la page de téléchargement a été refaite. Elle intègre désormais le responsive, des icônes par type de fichier, l'affichage de la taille du fichier, la date de dernière modification du fichier, la sélection multiple avec téléchargement groupé (bulk download), et une page d'authentification HTML remplaçant le popup natif du navigateur.

Workflow CI/CD: Un workflow GitHub Actions génère et publie automatiquement les deux artefacts à chaque tag de version.

4.2 Points positifs

- L'architecture MVC a pu être respectée, de nouveaux contrôleurs ont pu être ajoutés à l'application afin d'étendre les fonctionnalités en gardant des fichiers lisibles et compréhensibles : ``hotspot_controller.py``, ``network_controller.py`` et ``server_controller.py`` ont pu être ajoutés sans modifier le fonctionnement existant de l'application, conformément au principe de responsabilité unique.
- La décision de séparer ``server_controller.py`` de ``app_controller.py`` a permis de réutiliser directement la logique serveur dans le mode CLI sans duplication de code.
- La transparence sur l'utilisation de l'IA (commentaires ``# AI USE`` dans le code, mentions dans le journal de travail) garantit une traçabilité honnête des contributions de l'assistance de l'intelligence artificielle.
- Le pipeline CI/CD automatise entièrement le build, le packaging et la publication de l'application, ce qui permet d'accélérer le déploiement de la version à jour de l'application.

4.3 Difficultés particulières

La difficulté principale rencontrée a été un problème matériel : la fonctionnalité de point d'accès Wi-Fi repose sur ``netsh wlan hostednetwork``, qui exige une carte réseau supportant l'hébergement réseau. L'ordinateur fourni par le CPNV en était dépourvu, et plusieurs adaptateurs USB ont dû être testés avant de trouver un adaptateur compatible ce qui a retardé la validation de cette fonctionnalité de plus d'une semaine. Ce risque avait été identifié dans la matrice des risques (risque 1) et a été résolu grâce à l'aide reçue de M. Wulliamoz.

Une seconde difficulté a été le comportement différent entre l'environnement de développement et l'exécutable packagé. Plusieurs problèmes ont émergé et ont rendu la majorité des releases proposés lors du développement inutilisable ou incomplet. Voici les problèmes rencontrés :

- Transmission du chemin du répertoire en menu contextuel.
- Fenêtre de l'application s'ouvrant vide en mode CLI,
- Chemins de build incorrects

Ces deux derniers points ont pu être corrigés avant la fin du projet, mais l'ouverture via le menu contextuel est encore bloquant, représentant une régression dans le projet. Une solution mise en place a été de passer par Inno Setup pour l'écriture dans le registre qui est la manière commune de le faire mais le comportement n'a pas changé. Faute de temps pour investiguer davantage, cette limitation est documentée et constitue une priorité pour la version suivante.

4.4 Expérience d'apprentissage

Ce projet a permis d'approfondir plusieurs compétences techniques : la programmation asynchrone avec `asyncio` et `aiohttp`, la gestion des processus système sous Windows via `subprocess` et le registre Windows, ainsi que la mise en place d'un pipeline CI/CD complet avec GitHub Actions. La contrainte de développer une application Windows cible depuis un environnement Linux a également renforcé la rigueur dans la séparation du code en développement et penser au comportement en production.

La contrainte matérielle du point d'accès, bien qu'anticipée dans la matrice des risques, m'a confirmé l'importance de valider l'environnement d'exécution cible dès le début du projet d'une part pour éviter les problèmes techniques, et surtout, pour pouvoir mieux faire face aux imprévus afin de mieux pouvoir adapter le temps de résolution du problème. Sur le plan technique, travailler en développement Linux sur une cible Windows m'a appris à systématiquement distinguer le comportement en environnement de développement du comportement packagé, une rigueur que j'appliquerai dès le début sur tout projet similaire.

4.5 Perspectives d'amélioration

Plusieurs pistes d'amélioration ont été identifiées pour une version future de Share++ :

- Correction de la régression du menu contextuel : assurer la transmission correcte du chemin de dossier à l'exécutable packagé lors d'un lancement depuis l'Explorateur Windows.
- Ajout de la navigation dans les sous dossiers des répertoires partagés pour la page de téléchargement.
- Ajout d'étapes de build d'OS différents (macOS, Linux, Android, IOS) afin d'étendre la compatibilité de l'application.
- Ajout de la détection Tailscale et la création du hotspot ad hoc pour la version CLI de l'application
- Extension des tests automatisés aux nouvelles fonctionnalités de détection Tailscale, de refonte de la page de téléchargement et de l'interface CLI, dont la validation repose actuellement que sur des tests End-to-End manuels.

Si je recommençais un projet similaire, je serais plus prudent dans l'estimation du backlog initial. Ayant moi-même proposé les fonctionnalités du cahier des charges, j'ai eu tendance à sous-estimer leur temps d'implémentation. Le projet a abouti, mais au prix d'un rythme soutenu et grandissant durant le projet.

Ce rythme était aussi causé par un temps trop important accordé au débogage d'une fonctionnalité complémentaire ne faisant pas partie du cahier des charges, ce qui était une erreur. Au final, la promesse d'une release à jour du projet chaque semaine n'a même pas pu être tenue (la version proposée en fin de sprint comportait la plupart du temps un problème bloquant).

5 Annexes

5.1 Résumé du rapport du TPI

Contexte et objectif

Share++ est une application Windows conçue pour faciliter le partage de fichiers et de dossiers au sein d'un réseau local. Contrairement aux solutions traditionnelles dépendant de services tiers, du cloud, ou d'un support physique, cette application garantit la confidentialité des échanges en maintenant les données sur le réseau local de l'utilisateur et ses machines. Ce projet constitue une évolution de la version initiale (préTPI Share++ v1.0), intégrant cinq axes d'amélioration : la persistance des journaux, la détection du service Tailscale, la création d'un points d'accès Wi-Fi ad hoc, l'ajout d'une interface en ligne de commande et la refonte de la page de téléchargement.

Réalisation

L'application repose sur une architecture logicielle modulaire (MVC), permettant une maintenance plus simple pour de futurs ajouts et a permis l'ajout de nouvelles fonctionnalités sans dégradation du système existant. Des efforts sur l'intégration continue et le déploiement continu ont été mis en œuvre via la mise en place d'un pipeline GitHub Actions. Ce processus automatise la génération et la publication des versions de l'application, assurant ainsi la cohérence entre le code source et les livrables finaux, fonctionnel sur les deux versions de l'application.

Tests et qualité

Les fonctionnalités principales sont couvertes par des tests unitaires et d'intégration automatisés avec pytest. Les tests End-to-End ont été réalisés manuellement avec un smartphone Android comme client. La principale difficulté matérielle était l'absence de carte Wi-Fi compatible avec le mode réseau hébergé Windows. Elle a été documentée comme contrainte externe et couverte par des tests unitaires avec appels système simulés.

Conclusion

Les objectifs principaux du cahier des charges ont été atteints. L'application est fonctionnelle, installable et utilisable dans un contexte réel, aussi bien pour un utilisateur grand public que pour un utilisateur avancé via l'interface CLI. Les perspectives d'évolution identifiées (navigation dans les sous-répertoires, compatibilité multiplateforme) sont documentées dans les conclusions du rapport.

5.2 Sources

Flet

Framework UI python

- <https://flet.dev/docs>
- <https://flet.dev/gallery>

- <https://flet.dev/docs/publish/windows/>
- <https://flet.dev/docs/publish/>

aiohttp

Serveur http, refonte de l'interface graphique

- <https://docs.aiohttp.org/en/stable/index.html>
- https://docs.aiohttp.org/en/stable/web_quickstart.html#run-a-simple-web-server
- https://docs.aiohttp.org/en/stable/web_advanced.html#middlewares
- <https://github.com/feathericons/feather/tree/main/icons>

asyncio

Programmation asynchrone

- <https://docs.python.org/3/library/asyncio.html>

logging

- <https://docs.python.org/3/library/logging.html>
- <https://docs.python.org/3/howto/logging.html>
- <https://docs.python.org/3/library/logging.handlers.html>

netsh

- <https://github.com/DJStompZone/pyhotspot-win/blob/master/hotspot.py>
- <https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/netsh-wlan>

subprocess

Execution de commande sur la machine

- <https://docs.python.org/3/library/subprocess.html>

socket

détection des adresses IP de la machine

- <https://docs.python.org/3/library/socket.html>

re

Expressions régulières

- <https://docs.python.org/3/library/re.html>

pathlib

Gestion des chemins de fichiers/répertoires

- <https://docs.python.org/3/library/pathlib.html>

hashlib

Hash SHA-256

- <https://docs.python.org/3/library/hashlib.html>

qrcode

Génération des qr codes (GUI & CLI)

- <https://pypi.org/project/qrcode/>

shutil / tempfile

Compression .zip

- <https://docs.python.org/3/library/shutil.html>
- <https://docs.python.org/3/library/tempfile.html>

os

Actions générales sur le système

- <https://docs.python.org/3/library/os.html>

sys

paramètres de l'interpréteur

- <https://docs.python.org/3/library/sys.html>

platform

Détection de l'OS

- <https://docs.python.org/3/library/platform.html>

ctypes

Appels windows (élévations admin)

- <https://docs.python.org/3/library/ctypes.html>

base64 / io

encodage des QR codes :

- <https://docs.python.org/3/library/base64.html>
- <https://docs.python.org/3/library/io.html>

webbrowser

ouverture dans le navigateur

- <https://docs.python.org/3/library/webbrowser.html>

datetime

horodatage des logs

- <https://docs.python.org/3/library/datetime.html>

argparse

interface CLI

- <https://docs.python.org/3/library/argparse.html>
- <https://oneuptime.com/blog/post/2026-02-03-python-argparse-cli/view#choices-limiting-valid-values>
- <https://emojiterria.com/search/wrong/>

getpass

saisie sécurisée du mot de passe

- <https://docs.python.org/3/library/getpass.html>

Tailscale

Plage adresses IP Tailscale et Tailscale CLI

- <https://tailscale.com/docs/concepts/tailscale-ip-addresses>
- <https://tailscale.com/docs/reference/tailscale-cli?tab=windows>

Workflow CI/CD

- https://dev.to/rahul_suryash/steps-to-build-binary-executables-for-python-code-with-github-actions-4k92
- <https://github.com/actions/setup-python>

PyInstaller

- <https://pyinstaller.org/en/stable>

Inno Setup

- <https://jrsoftware.org/ishelp/>
- https://documentation.help/Inno-Setup/topic_registrysection.htm
- <https://stackoverflow.com/questions/3304463/how-do-i-modify-the-path-environment-variable-when-running-an-inno-setup-install>

Tests

- <https://docs.pytest.org/en/stable/>
- <https://pytest-asyncio.readthedocs.io/en/latest/>
- <https://docs.python.org/3/library/unittest.mock.html>

Utilisation de l'intelligence artificielle

Durant ce projet, des outils d'IA (Gemini) ont été utilisés ponctuellement comme aide au débogage et à la compréhension, principalement lors de problèmes bloquants : contraintes de privilèges Windows (CREATE_NO_WINDOW, élévation UAC), comportement différents entre l'environnement de développement et l'exécutable packagé, et stratégie de test pour les fonctionnalités dépendantes du matériel.

Dans tous les cas, le code proposé par l'IA a été relu, compris et adapté avant intégration. Aucun bloc de code n'a été copié-collé sans analyse préalable. Les contributions de l'IA sont systématiquement signalées par des commentaires `# AI USE` ou `# AI FIX` dans le code, et avec une description de la nature de l'aide apportée. L'utilisation de l'IA a également été mentionnée scrupuleusement dans le journal de travail.

5.3 Glossaire

Terme	Définition
Ad hoc	Mode réseau sans fil dans lequel les appareils communiquent directement entre eux, sans passer par un point d'accès centralisé.
aiohttp	Librairie Python permettant de créer des serveurs HTTP asynchrones, basée sur asyncio.
API	Application Programming Interface. Interface permettant à deux logiciels de communiquer entre eux selon un contrat défini.
argparse	Module Python de la librairie standard permettant de définir et parser (analyser/traiter) les arguments en ligne de commande.
asyncio	Librairie Python de la librairie standard permettant d'écrire du code concurrent à l'aide de coroutines et d'une boucle d'événements.
asyncio.to_thread()	Fonction asyncio permettant d'exécuter une fonction bloquante dans un thread séparé sans bloquer la boucle d'événements principale.
Bulk download	Un bulk download (téléchargement en masse) est un téléchargement de plusieurs fichiers en une seule opération.
CGNAT	Carrier-Grade NAT. Technique de translation d'adresses réseau utilisée par les opérateurs. Tailscale utilise la plage 100.64.0.0/10 qui en fait partie.
CI/CD	Continuous Integration / Continuous Deployment. Philosophie d'automatisation de certaines tâches redondantes à chaque modification du code. Accélérant l'intégration et le déploiement continu.

CLI	Command Line Interface. Interface en ligne de commande permettant d'interagir avec un programme via un terminal, sans interface graphique.
Cookie de session	Données stockées dans le navigateur du client pour authentifier la session de l'utilisateur.
CREATE_NO_WINDOW	Flag Windows (valeur 0x08000000) passé à subprocess pour empêcher l'apparition d'une fenêtre de terminal lors de l'exécution d'une commande.
CWD	Current Working Directory. Répertoire de travail courant depuis lequel un processus est lancé.
DoD	Definition of Done. Critères explicites qu'une user story doit satisfaire pour être considérée comme terminée dans un projet Scrum.
Exécutable autonome	Fichier .exe embarquant toutes ses dépendances (runtime Python, Flutter, librairies) et pouvant s'exécuter sans installation préalable.
ExpansionTile	Widget Flet affichant un en-tête cliquable qui révèle ou masque un contenu supplémentaire.
Flet	Framework Python basé sur Flutter permettant de créer des interfaces graphiques natives multiplateformes en Python.
Flutter	Framework UI de Google utilisé par Flet comme moteur de rendu.
Framework	Ensemble cohérent d'outils, de conventions et de bibliothèques facilitant le développement d'applications dans un domaine donné.
getpass	Module Python permettant de saisir un mot de passe en ligne de commande sans l'afficher à l'écran.
GitHub Actions	Service d'intégration et déploiement continu intégré à GitHub, permettant d'automatiser des workflows déclenchés par des événements Git.
HMAC-SHA256	Hash-based Message Authentication Code utilisant SHA-256. Utilisé ici pour signer les cookies de session et garantir leur intégrité.
Hotspot	Point d'accès Wi-Fi permettant à d'autres appareils de se connecter à Internet ou à un réseau local via une machine hôte.
Headless	Mode d'exécution d'une application sans interface graphique, utilisation uniquement en ligne de commande.
HMAC	Hash-based Message Authentication Code. Mécanisme cryptographique garantissant l'authenticité et l'intégrité d'un message.

Inno Setup	Outil open-source permettant de créer des installateurs Windows professionnels (.exe).
IPv4	Internet Protocol version 4. Format d'adresse réseau sur 32 bits (ex. 192.168.1.1).
LAN	Local Area Network. Réseau local physique, typiquement un réseau domestique ou d'entreprise.
Logging	Journalisation. Mécanisme d'enregistrement d'événements applicatifs dans un fichier ou une console, avec niveaux de sévérité (INFO, WARNING, ERROR).
Middleware	Middleware (inter-logiciel) est un logiciel intermédiaire entre la requête et la réponse effectuant du traitement.
MIT	Licence logicielle permissive autorisant l'utilisation, la modification et la redistribution du code, y compris dans des projets commerciaux.
MVC	Model-View-Controller. Patron d'architecture logicielle séparant les données (Model), l'interface (View) et la logique métier (Controller).
NAT	'Network Address Translation' est une technique qui permet à plusieurs appareils d'un réseau privé de partager une seule adresse IP publique pour accéder à Internet. Elle modifie les adresses IP des paquets de données lors de leur transit à travers un routeur.
NAT traversal	Technique permettant à deux machines situées derrière des routeurs NAT différents de communiquer directement, sans redirection de port manuelle.
netsh	Network Shell. Interpréteur de commandes Windows permettant de configurer les interfaces réseau, dont la création de réseaux hébergés Wi-Fi.
Overlay Network	Réseau virtuel superposé à un réseau physique existant. Tailscale crée un tel réseau entre ses clients.
PyInstaller	Outil Python permettant de packager un script Python et toutes ses dépendances en un exécutable autonome.
pytest	Framework de tests Python populaire, utilisé pour écrire et exécuter des tests unitaires et d'intégration.
QR Code	Quick Response Code. Code-barres bidimensionnel encodant une information (ici une URL) lisible par une caméra de smartphone.
Regex	Expression régulière (Regular Expression). Séquence de caractères définissant un motif de recherche dans du texte.
Responsive Design	Conception d'interface web s'adaptant automatiquement à la taille de l'écran de l'appareil (mobile, tablette, bureau).

RotatingFileHandler	Handler de la librairie logging Python créant automatiquement un nouveau fichier de log lorsque la taille maximale est atteinte.
Scrum	Méthode de gestion de projet agile découpant le travail en sprints itératifs avec des livrables à chaque fin de cycle.
SHA-256	Secure Hash Algorithm 256 bits. Fonction de hachage cryptographique utilisée pour stocker les mots de passe de manière plus sécurisée.
Sprint	Itération de durée fixe dans la méthode Scrum, à l'issue de laquelle un incrément fonctionnel est livré.
subprocess	Module Python permettant de lancer des processus système et de récupérer leur sortie.
SVG	Scalable Vector Graphics. Format d'image vectorielle.
Tailnet	Réseau privé virtuel constitué de tous les appareils d'un même compte Tailscale.
Tailscale	Service de réseau maillé sécurisé basé sur WireGuard.
TPI	Travail Pratique Individuel. Travail final de validation du CFC.
UAC	User Account Control. Mécanisme Windows demandant une confirmation ou des identifiants administrateur avant d'exécuter des actions sensibles.
User-Agent	En-tête HTTP envoyé par le navigateur ou l'application cliente, identifiant le type d'appareil et le système d'exploitation.
User Story	Unité de travail décrivant une fonctionnalité pour l'utilisateur final.
WireGuard	Protocole VPN moderne, rapide et sécurisé, sur lequel repose Tailscale.
WPA2	Wi-Fi Protected Access 2. Standard de sécurité Wi-Fi exigeant un mot de passe d'au moins 8 caractères.

5.4 Journal de travail

JOURNAL DE TRAVAIL — PROJET TPI — Share++						
Créé le 23/04/2026						
Date	Heure début	Heure fin	Durée (h)	Type	Libellé	Notes
23-04-2026	8:30	9:00	0.50h	Réunion	Démarrage du TPI, Reception du cahier des charges	
23-04-2026	9:00	9:50	0.83h	Doc	Rédaction du planning initial en Gantt	
23-04-2026	10:15	10:30	0.25h	Réunion	Entretien avec Chef de projet	
23-04-2026	10:30	11:00	0.50h	Doc	Rédaction de l'analyse Access Point	pywifi est dépréciée
23-04-2026	11:00	11:50	0.83h	Doc	Suppression des anciens documents préTPI et mise en place des répertoires de documentation	
23-04-2026	11:50	12:30	0.67h	Doc	Analyse et conception Access Point	utiliser netsh, car natif windows
27-04-2026	8:15	9:15	1.00h	Doc	Réécriture de l'analyse préliminaire	
27-04-2026	9:15	9:50	0.58h	Doc	Ajout d'issues github par rapport aux objectifs rédigés du cahier des charges	

27-04-2026	10:10	10:55	0.75h	Code	Implémentation des logs persistents (persistance du fichier, écriture du fichier, initialisation du logger et handler)	J'utilise la librairie de base python 'logging' https://docs.python.org/3/howto/logging.html
27-04-2026	10:55	11:45	0.83h	Code	Implémentation des logs persistents (formatter)	
27-04-2026	11:45	12:30	0.75h	Code	Ajout d'un workflow censé pouvoir build et packager lors de push avec tag de version	
27-04-2026	13:20	13:30	0.17h	Code	Test du workflow et fix des problèmes survenants	winreg à du être enlevé des requirements.txt, passer à python 3.12
27-04-2026	13:30	13:45	0.25h	Code	Regler les problèmes du workflow 1 par 1	Step : build with flet, UnicodeEncodeError, solution trouvée : ajouter PYTHONUTF8 env var to workflow
27-04-2026	13:45	14:00	0.25h	Code	Regler les problèmes du workflow 1 par 1	Step : build with flet Problème : flutter 3.41.4 required Solution : Ajouter un step d'installation de flutter
27-04-2026	14:00	14:20	0.33h	Code	Regler les problèmes du workflow 1 par 1	Step : Update version in setup.iss Problème : setup.iss absent à l'endroit cherché Solution : Déplacer le setup.iss, enlever les icones car elles ne sont pas essentielles.

27-04-2026	14:20	14:30	0.17h	Code	Regler les problèmes du workflow 1 par 1	Step : Create Github release Problème : github token does not have required permission to create a release Solution : Settings->Actions->General->WorkflowPermissions->Allow read and write permissions.
27-04-2026	14:30	14:55	0.42h	Code	Regler les problèmes du workflow 1 par 1	Workflow OK, app crash 'no module named qrcode'. Solution : copier le requirements.txt dans src/ (la ou InnoSetup cherche les fichiers pour build à son tours).
27-04-2026	15:10	15:40	0.50h	Test	Adaptation du plan de test du préTPI	
27-04-2026	15:40	16:40	1.00h	Doc	Documentation de la fonctionnalité avancée du logging (section 3.3)	
28-04-2026	8:15	8:45	0.50h	Doc	Rédaction de la section 2.1 du dossier de projet	des parties ont été reprise du projet préTPI
28-04-2026	8:45	9:50	1.08h	Doc	Rédaction des sections 2.2 et 2.3 du dossier de projet	des parties ont été reprise du projet préTPI
28-04-2026	10:05	10:40	0.58h	Autre	Maquette de l'interface de l'application windows	

28-04-2026	13:20	14:00	0.67h	Code	Implémentation du contrôleur hotspot dédié 'hotspot_controller.py'	Alternative analysée : PyNetsh, mais il semble déprécié depuis 7ans. Ressources utilisées : Intelligence artificielle (Gemini) J'ai trouvé très peu de ressources sur le sujet, j'ai donc questionné l'IA sur l'utilisation de netsh sur python puis j'ai également récupéré des morceaux de code trouvés sur github https://github.com/DJStompZone/pyhotspot-win/blob/master/hotspot.py
28-04-2026	14:00	14:50	0.83h	Code	Implémentation des méthodes hotspot_controller	
28-04-2026	15:05	15:50	0.75h	Autre	Maquette de l'interface de l'application windows	Ajout d'une carte dédiée pour l'access point
28-04-2026	15:50	16:40	0.83h	Autre	Maquette de l'interface de l'application windows	Amélioration design et interface de téléchargement
29-04-2026	8:20	9:00	0.67h	Autre	Maquette de l'interface de l'application windows	
29-04-2026	9:00	9:50	0.83h	Code	Implémentation du code access point non testé	
29-04-2026	10:10	10:50	0.67h	Code	Ajout d'une card dédiée au controle du hotspot dans view.py	

29-04-2026	10:50	11:30	0.67h	Code	Regler un problème de bouton	Problème rencontré : Le bouton stop server est inactif et ne peut pas être cliqué. On ne peut donc plus arrêter le serveur. Solution trouvée : control_card ne doit pas désactiver la toggle_card complète, mais simplement désactiver 1 a 1 les widgets.
29-04-2026	11:30	12:30	1.00h	Test	Ajout de tests spécifiques aux fonctionnalités à ajouter afin de pouvoir rapidement tester mon app plus tard. (plan)	Reprise du plan de test du préTPI. Ajout de test spécifiques aux nouvelles fonctionnalités. Premières entrées dans le journal de test
29-04-2026	13:20	13:50	0.50h	Test	Implémentation des test unitaires	
29-04-2026	13:50	14:50	1.00h	Doc	Description du travail effectué + environnement de travail	Section 3.3
29-04-2026	15:05	15:50	0.75h	Doc	Description du travail effectué + environnement de travail	
29-04-2026	15:50	16:40	0.83h	Test	Implémentation du reste des tests	Je me suis basé sur la syntaxe de tests précédentes et ai utilisé Gemini pour la création d'une fonction
30-04-2026	8:25	9:00	0.58h	Test	Fix des tests cassés	Changement de nom du constructeur était bloquant et empêchait le test de pass.
30-04-2026	9:00	9:50	0.83h	Test	Journalisation des tests	

30-04-2026	10:05	10:30	0.42h	Test	Journalisation des tests	
30-04-2026	10:30	11:00	0.50h	Réunion	Sprint review 1	
30-04-2026	11:00	12:30	1.50h	Doc	rédaction de l'email de fin de sprint. Export des documents pour le push final	
04-05-2026	8:20	8:40	0.33h	Code	Fix le problème de workflow : problème : step setup-python outdated. J'ai utilisé setup-python v6 avec python 3.13 comme recommandé ici : https://github.com/actions/setup-python	
04-05-2026	8:40	9:00	0.33h	Code	Access Point, interface réseau manquant	Clé Wi-Fi reçue, problème rencontré : Même après installation des drivers, la fonctionnalité de réseau hébergé n'est pas prise en charge par la clé USB. - Solution trouvée : j'abandonne le test de cette fonctionnalité. Elle est malheureusement laissée là sans plus qu'on n'y touche jusqu'à ce qu'une nouvelle clé me soit parvenue. Je n'en commande pas de nouvelle.
04-05-2026	9:00	9:20	0.33h	Doc	Documentation du problème d'interface Wi-Fi manquant	

04-05-2026	9:35	10:00	0.42h	Autre	Maquette de l'interface, comprenant la fonctionnalité d'affichage de plusieurs qr code.	Problème rencontré : Afficher beaucoup de qr code dans l'application devient vite moche, peu pratique et encombrant. Solution trouvée : Utiliser ft.ExpansionTile afin de pouvoir lister les adresses et permettre pour chacune d'afficher le qr.
04-05-2026	10:00	10:30	0.50h	Code	Implémentation de l'affichage de listage d'adresses en ExpansionTile	
04-05-2026	10:30	11:00	0.50h	Code	Changement de la manière dont les IP sont scannées	Ressources : https://tailscale.com/docs/concepts/tailscale-ip-addresses
04-05-2026	11:00	11:45	0.75h	Autre	Maquette de l'interface	Mise à jour des maquettes pour combiner la fonctionnalité de plusieurs qr code à celle de création de hotspot au niveau visuel
04-05-2026	11:45	12:30	0.75h	Code	Fonctionnalité Tailscale	Ajout d'une détection tailscale par retour de commande tailscale CLI (tailscale ip -4) + test de la connexion validant le fonctionnement pour l'utilisateur
04-05-2026	13:20	14:00	0.67h	Code	Fonctionnalité Tailscale	
04-05-2026	14:00	14:50	0.83h	Doc	Documentation de la fonctionnalité avancée de l'implémentation Tailscale	

04-05-2026	15:05	15:30	0.42h	Doc	Documentation de la fonctionnalité avancée de l'implémentation Tailscale	
04-05-2026	15:30	16:30	1.00h	Test	Test de l'implémentation Tailscale	Problème rencontré : Le workflow rencontre des problèmes après l'ajout du pyproject.toml, Solution trouvée : debug le workflow step by step
05-05-2026	08:20	09:00	0.67h	Autre	Debug Workflow	Problème rencontré : une release a pu être créée par le workflow, mais l'application quitte après le lancement. Le fichier ne fait plus que 15Mo (contre 45Mo) Solution trouvée : revenir à un workflow précédent qui fonctionnait
05-05-2026	09:00	09:30	0.50h	Autre	Debug Workflow	Le problème à bien été résolu après avoir introduit des prints de debug pour afficher l'état des fichiers après le build de flet. les dossiers DLLs/, Lib/ et site-packages/ n'étaient pas copiés, réduisant l'exé à 15Mo au lieu de 45Mo et rendant l'application inutilisable.
05-05-2026	09:30	09:50	0.33h	Test	Test de l'implémentation Tailscale	L'implémentation Tailscale semble en ordre. La disponibilité de l'adresse de manière dynamique est aussi validée

05-05-2026	09:50	10:20	0.50h	Code	Implémentation de la fonctionnalité CLI	Implémentation de l'interface TUI (--nogui) : création de cli.py, fichier dédié à la fonctionnalité CLI Guide : https://oneuptime.com/blog/post/2026-02-03-python-argparse-cli/view#choices-limiting-valid-values
05-05-2026	10:20	10:50	0.50h	Réunion	Entretien avec le deuxième expert	
05-05-2026	13:20	14:00	0.67h	Code	Implémentation de la fonctionnalité CLI	Traitement des arguments avec argparse, création du QR code ASCII avec la même librairie 'qrcode' utilisée pour le rendu qr GUI. logs terminal colorés. Découplage de main.py pour séparer la détection GUI/CLI du lancement Flet.

05-05-2026	14:00	14:55	0.92h	Code	Implémentation de la fonctionnalité CLI	Refactoring : Création d'un fichier server.py dédié à la logique serveur aiohttp, utilisé pour l'app GUI et TUI, afin d'éviter la duplication du code. Déplacement de parse_user_agent() vers server.py. Utilisation de l'IA : Gemini m'a aidé à réaliser mon idée en me proposant de passer une fonction log différente à la fonction build_app de server.py. app_controller.py et cli.py appellent désormais build_app() Emojis : https://emojiterra.com/search/wrong/
05-05-2026	15:05	15:30	0.42h	Code	Implémentation de la fonctionnalité de changement de port par l'utilisateur	Ajout du changement de port persistant dans les paramètres de l'application flet via un AlertBox. La valeur est bien sauvegardée dans settings.json
05-05-2026	15:30	16:00	0.50h	Code	Ajout du PATH Windows dans setup.iss via section [Registry] et fonction Pascal NeedsAddPath(). Permet d'utiliser sharepp depuis n'importe quel terminal sans chemin absolu.	guide suivi : https://flet.dev/docs/publish/ https://documentation.help/Inno-Setup/topic_registrysection.htm https://stackoverflow.com/questions/3304463/how-do-i-modify-the-path-environment-variable-when-running-an-inno-setup-install
05-05-2026	16:00	16:40	0.67h	Code	Documentation du travail effectué	

06-05-2026	08:20	08:40	0.33h	Test	Réception du laptop Windows CPNV et constat que la fonctionnalité Hosted Network n'est pas supportée non plus (netsh wlan show drivers : non). Push du code bien commit de la veille et push d'un nouveau tag de release	Deuxième machine consécutive sans support. Fonctionnalité hotspot définitivement non validable sur le matériel disponible. Sera documenté comme problème rencontré durant le projet.
06-05-2026	08:55	09:10	0.25h	Autre	Test de plusieurs adaptateurs USB avec M. Wuliamoz jusqu'à trouver un adaptateur compatible.	
06-05-2026	09:10	09:50	0.67h	Test	Découverte et diagnostic du bug d'erreur 500 du serveur aiohttp : aucun log de connexion ou de warning, tentative d'accès sans pare-feu, sur localhost...	Utilisation de l'IA (Gemini) pour m'aider à diagnostiquer le problème. Selon lui, le type d'event loop asyncio peut-être la cause. Il faudrait passer de ProactorEventLoop à SelectorEventLoop.
06-05-2026	10:05	10:50	0.75h	Code	Implémentation des résolutions des problèmes identifiés	

06-05-2026	10:50	11:30	0.67h	Autre	Test de plusieurs Adaptateur Wireless USB avec M. Wulliamoz jusqu'à trouver une supportant l'hébergement réseau.	Finalement, la LogiLink, adaptateur USB view de plus de 13 ans a fonctionné. L'application a également pu créer un réseau ad-hoc, mais le flux doit être revu, car il faut inclure de manière évidente le lien de partage issu du réseau ad-hoc.
06-05-2026	11:30	12:30	1.00h	Autre	Maquette de l'interface, modification de l'interface servant à entrer le SSID/mdp pour le Wi-Fi	Il n'est pas logique pour l'utilisateur de devoir gérer indépendamment le réseau ad-hoc et le partage de fichier. Je reprends la logique de l'activation du mode comme pour la protection par mdp. Le hotspot doit être un paramètre que l'on peut activer ou désactiver à sa guise facilement comme la protection par mdp
06-05-2026	13:20	14:00	0.67h	Code	Modification de view.py pour appliquer les changements de la maquette, intégration du hotspot dans l'interface : suppression de la card hotspot indépendante, intégration dans la card principale via un Chip persistant et un dialog de configuration.	Le hotspot démarre et s'arrête automatiquement avec le serveur. Chip grisé sur Linux.

06-05-2026	14:00	14:30	0.50h	Code	Ajout de <code>get_hotspot_ip()</code> dans <code>network_controller.py</code> pour détecter l'adresse du hotspot. Refactor de <code>show_connection_tiles()</code> , suppression de <code>show_qr_code()</code> et <code>show_dual_qr()</code> . La fonction gère désormais LAN, Tailscale et hotspot en une seule méthode.	
06-05-2026	14:30	14:50	0.33h	Code	Modification dans la détection Tailscale : vérification du retour de code de <code>tailscale status</code> .	https://tailscale.com/docs/reference/tailscale-cli?tab=windows
06-05-2026	15:05	15:50	0.75h	Code	Correction du bug de logs après redémarrage du serveur : remplacement de <code>page.run_thread</code> par <code>page.run_task</code> dans <code>Controller.log()</code> .	https://flet.dev/docs/controls/page/
06-05-2026	15:50	16:40	0.83h	Code	Retirer l'argument <code>-password</code> car demander le mdp en clair est une erreur de cybersécurité car il serait récupérable facilement dans l'historique de commandes.	

07-05-2026	08:20	09:00	0.67h	Doc	Rédaction de la documentation : sections 3.3.3, 3.3.4, 3.3.5 mises à jour, UML des classes mis à jour, arborescence du projet actualisée.	
07-05-2026	09:00	09:10	0.17h	Test	Test de l'application, problème rencontré : le bouton configurer le réseau Wi-Fi n'ouvre pas l'alertdialog de configuration. Doublement des logs à la création du hotspot, la création de point d'accès fait freeze l'app.	
07-05-2026	09:10	09:50	0.67h	Code	Implémentation des fix pour la version packagée.	J'ai utilisé l'IA de manière conséquente pour m'aider à régler ces problèmes car ils étaient en dehors de mes connaissances. J'ai toutefois bien documenté les modifications dans le code et ai essayé de comprendre ce que j'ajoutais
07-05-2026	10:05	11:10	1.08h	Code	Implémentation des fix pour la version packagée.	Plusieurs itérations d'implémentations des fix ont été nécessaires, accompagné de plusieurs releases (1.2.17, 1.2.18, 1.2.19)
07-05-2026	11:10	11:30	0.33h	Réunion	Entretien de fin de sprint 2	

07-05-2026	11:30	12:30	1.00h	Autre	Mise à jour du dépôt Github, des documents, et écriture du mail de fin de sprint	
11-05-2026	08:15	09:00	0.75h	Code	Test de la version packagée de l'app (v1.3.0).	Problème rencontré : plus aucun widget visible. Le bouton contextuel affiche toujours access denied. Le cli ouvre toujours une fenêtre vide et le terminal reste vierge. Solution trouvée : Retirer le code lié à l'implémentation du menu contextuel pour utiliser la solution plus classique d'écrire les clés de registres via l'assistant d'installation (modifier setup.iss). Ajouter un step au workflow pour packager une version de sharepp_cli.exe plus légère que l'app GUI (variante numéro 2 analysée).
11-05-2026	09:00	09:50	0.83h	Code	Implémentation des changements	Modification de main.py pour rendre cli standalone. Validation du bon fonctionnement en environnement de dev. Ajout d'un step au workflow pour packager la version cli avec pyinstaller.

11-05-2026	10:05	11:00	0.92h	Code	Implémentation des changements	Modification de setup.iss pour inclure l'écriture dans le registry windows. Enlever le doublons dans le code (il n'est plus possible d'activer / désactiver le bouton contextuel sans désinstaller l'application)
11-05-2026	11:00	12:30	1.50h	Code	Documentation des sections 2.4 et 2.5	Ajout de l'UML qu'il faudra encore mettre à jour pour la version finale.
11-05-2026	13:20	14:00	0.67h	Code	Tentative workflow, lancement du programme depuis le poste windows pour obtenir les message de debug plus rapidement.	Problème rencontré : Une mise à jour du code avait rendu les widgets invisibles sans faire crash l'application. Impossible de savoir que le programme crashait dès le départ. Solution trouvée : Développer depuis l'ordinateur Windows directement à permi de débloquent la situation. (message d'erreur présent sur windows mais absent sur Linux.) (méthodes toujours utilisées dans app_controller mais ayant été supprimée de view.py.). Bizarrement le problème n'était pas bloquant sur l'environnement de développement Linux.
11-05-2026	14:00	14:50	0.83h	Code	Implémentation de tous les fix et création des dernières maquettes	

11-05-2026	15:05	16:00	0.92h	Code	Avancement sur l'implémentation de la refonte de la download page	Améliorations : ajout de propriété de date de dernière modif avec <code>os.path.getmtime</code> et la taille avec <code>os.path.getsize</code> pour chaque fichier
11-05-2026	16:00	16:40	0.67h	Code	Fix workflow incrémentiel + dispatch de <code>download_view</code>	Fix incrémentiel du workflow (prend 10min à chaque fois). En attendant : dispatch du js et du css dans un répertoire statique servi par aiohttp : <code>app.router.add_static</code> .
12-05-2026	08:15	09:00	0.75h	Code	Fix workflow incrémentiel + ajout d'une page html de connexion	Le but est de remplacer le basic auth header ayant forcément un champ 'utilisateur' par une page html de connexion custom.
12-05-2026	09:00	09:50	0.83h	Code	Implémentation de la nouvelle page de connexion	Ajout d'une fonction <code>generate_auth_html</code> (page de connexion) en dessous du <code>generate_html</code> (page de téléchargement)
12-05-2026	10:05	10:50	0.75h	Code	Style de la page de téléchargement & connexion	Fichier unique <code>download.css</code> pour les deux pages html, question de simplicité. Ajout des icônes svg pour chaque type de fichier. Source : https://github.com/feathericons/feather/tree/main/icons
12-05-2026	13:20	14:00	0.67h	Autre	Fix du workflow, fix du CLI	Le fix ne fonctionne toujours pas. Je reviendrai à une version passée fonctionnelle en cas d'échec sur la prochaine itération de fix.

12-05-2026	14:00	14:50	0.83h	Code	Ajout de la fonctionnalité download history	Ajout du parsing de sharepp.log afin de récupérer uniquement les téléchargements et leurs métadonnées avec une expression régulière. Ajout d'un bouton d'affichage de la page d'historique dans le popmenubutton
12-05-2026	15:05	15:50	0.75h	Autre	Création de maquette	Création de la maquette de l'interface de l'historique pour l'application sur PenPot
12-05-2026	15:50	16:40	-	Absence	Malentendu organisationnel	Absence (les détails de cette absence ont été détaillé dans un mail envoyé aux experts ainsi qu'au chef de projet.) Ce temps perdu sera compensé par des tests utilisateurs passés en dehors des heures de cours à des personnes tierces (amis, famille)
13-05-2026	08:20	09:00	0.67h	Code	Fix workflow	
13-05-2026	09:00	09:50	0.83h	Code	Fix workflow & développement de l'interface d'historique des téléchargements	Ajout et finition du AlertDialog d'affichage des téléchargements
13-05-2026	10:05	10:50	0.75h	Doc	Mise à jour du dossier de projet	Documentation de la fonctionnalité CLI (changement de variante retenue)
13-05-2026	10:50	11:30	0.67h	Doc	Mise à jour du dossier de projet	Ajout des sections 4.1, 5.1
13-05-2026	11:30	12:30	1.00h	Doc	Mise à jour du dossier de projet	Ajout de la section 5.2

13-05-2026	13:20	14:00	0.67h	Doc	Mise à jour du dossier de projet	Ajout des sources
13-05-2026	14:00	14:50	0.83h	Code	Debug Workflow	AI USE : J'ai continué d'essayer de debug le workflow avec l'aide de l'IA. Au final, Il reste l'AppBar bleue. La piste à privilégier pour régler le bug est probablement de regarder dans le code ajouté récemment car rien dans le workflow semble être la cause
13-05-2026	15:05	15:50	0.75h	Doc	Mise à jour du dossier de projet	Ajout des sources
13-05-2026	15:50	16:40	0.83h	Code	Debug Workflow	Tentative de trouver dans le code l'erreur ajoutée parmi les nouvelles fonctionnalité
18-05-2026	08:15	08:45	0.50h	Code	Implémentation de prévention sur l'affichage des widgets	Ajout de la vérification prévenant contre l'affichage d'éléments non fonctionnels.
18-05-2026	08:45	09:50	1.08h	Doc	Rédaction du dossier de projet	Ajout du glossaire
18-05-2026	10:05	10:30	0.42h	Doc	Rédaction du dossier de projet	Ajout du glossaire
18-05-2026	10:30	10:45	0.25h	Réunion	Sprint review 3	

18-05-2026	10:45	12:30	1.75h	Doc	Rédaction du guide d'installation, correctifs du workflow	Correctifs du workflow. (Utilisation de l'IA : J'ai utilisé l'intelligence artificielle pour déboguer le workflow, en le questionnant sur les problème du step d'Inno Setup qui ne prend plus toutes les dépendances pour le packaging. Problème au niveau du chemin de fichiers setup.iss et du nom de l'exécutable principal.) Rédaction du guide d'installation en markdown
18-05-2026	13:20	14:00	0.67h	Doc	Rédaction du guide d'utilisation	Rédaction du guide d'utilisation en markdown
18-05-2026	14:00	14:50	0.83h	Doc	Rédaction du guide d'utilisation	Screenshots de l'application, rédaction du doc
18-05-2026	15:05	15:50	0.75h	Doc	Rédaction du README	
18-05-2026	15:50	16:40	0.83h	Doc	Rédaction du README	ajout de la section sur la couverture de test dans le dossier de projet
19-05-2026	08:15	09:00	0.75h	Doc	Rédaction de justification des choix	Justification des choix des technologies utilisées
19-05-2026	09:00	09:50	0.83h	Doc	Rédaction des sections tests	
19-05-2026	10:05	10:50	0.75h	Doc	Fusion des pdf	Fusion des pdf de journal de bord, de travail, de tests...

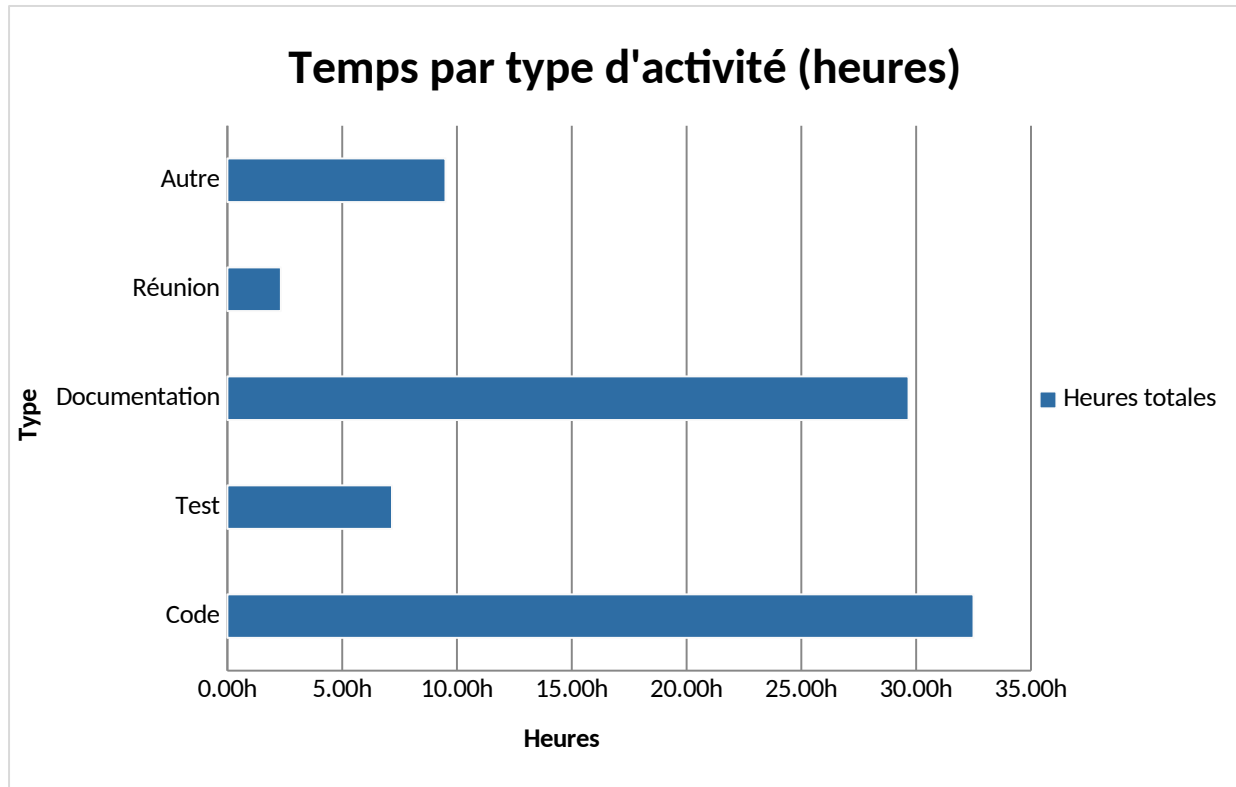
19-05-2026	13:20	14:00	0.67h	Doc	Documentation du test utilisateur	Documentation du test utilisateur réalisé la veille dans la documentation et le journal de test. Dernières modification dans la doc
19-05-2026	14:00	14:50	0.83h	Doc	Ajout des journaux	
19-05-2026	15:05	16:00	0.92h	Doc	Ajout des guides	
19-05-2026	16:00	16:40	0.67h	Doc	Mise en page	
20-05-2026	08:15	09:50	1.58h	Doc	Reliure	
TOTAL HEURES			82.50h			

5.5 Journal de bord

Type d'entretien	Date et heure	Personnes présentes	Sujet	Décisions
Entretien de début de projet	2026-04-23, 8h30-9h	Alexandre Ramirez, Gaëtan Gendroz, Xavier Carrel (+1 enseignant), Nathan Filipowitz	Validation du cahier des charges	Le cahier des charges a été signé. Le support matériel CD n'est pas obligatoire. Il faut livrer par mail hebdomadaire (jeudi) : journal de travail, dossier de projet (ou les trouver sur github) et un lien vers le repo. Les documents doivent être au format PDF. Des questions non spécifiques au projet pourront être posé pendant la présentation. Pseudo Github : xcarrel, il faut demander le pseudo du deuxième expert dans le mail d'aujourd'hui. Il ne faut pas copier coller le cahier des charges pour remplir le dossier de projet.
Entretien de début de projet	2026-04-23, 10h15-10h30	Pascal Benzonana, Nathan Filipowitz	Lancement du projet	Il y a beaucoup de fonctionnalités, je décide de m'y mettre le plus tot possible. Je décide de traduire les fonctionnalités de mon application en issue github.
Sprint review 1	2026-04-30, 10h30-11h	Pascal Benzonana, Nathan Filipowitz	Sprint review	- Lorsque l'on présente des technologies ou des solutions, il faut parler des variantes et motiver son choix. Pas simplement dire que l'on utilise tel ou tel solution. - Le travail réalisé est satisfaisant

Entretien avec le deuxième expert	2026-05-05, 10h20-10h50	Guillaume Blanco, Nathan Filipowicz	Etat de l'avancement du projet	Envoyer un mail au chef de projet pour communiquer le problème persistant lié au matériel Chercher la date exacte de la présentation + faire un doodle si la date n'est pas imposée Après renseignement de M. Benzonana : la présentation aura lieu le 03 juin à 15h15 dans la salle 214.
Sprint review 2	2026-05-07, 11h10-11h30	Pascal Benzonana, Nathan Filipowicz	Sprint review	- Il est OK d'utiliser l'IA pour régler les problèmes survenant après le packaging de l'application. Ce sont des fonctionnalités implémentée avec peu d'IA, mais régler ces problèmes se trouve être en dehors de mes compétences actuelles. Il faut toujours bien documenter toute son utilisation de l'IA. - Je reprends les stories restantes des fonctionnalités pas totalement implémentées au prochain sprint (CLI & hotspot adhoc.).
Sprint review 3 (visioconférence)	2026-05-18, 10h30-10h45	Pascal Benzonana, Nathan Filipowicz	Sprint review	'- Il faut vérifier le nombre de sections maximales pour le résumé du TPI dans les critères d'évaluations - Le chef de projet ne sera peut-être pas présent le jour de la présentation, (ou visioconférence) - Je dois cloturer le github project

5.6 Récapitulatif



RÉCAPITULATIF PAR TYPE D'ACTIVITÉ (Le TOTAL est inexacte)

Code	32.50h	40.0%
Test	7.17h	8.8%
Documentation	29.67h	36.6%
Réunion	2.33h	2.9%
Autre	9.50h	11.7%
TOTAL	81.17h	100.0%

5.7 Plan de tests

ID	Titre	Procédure	Résultat attendu	Type
TU-01	Persistence JSON	appeler test_model_initialization, Pour appeler model.save_settings()	Un nouveau fichier settings.json est recréé avec les valeurs par défaut	Unitaire

TU-02	Hashage password	définir un mot de passe avec <code>model.save_password()</code> , Vérifier la donnée sauvegardée avec <code>model.check_password()</code>	mdp hashé, validation retourne True pour un bon mdp et False pour un mauvais	Unitaire/ Sécurité
TU-03	Génération QR	à <code>generate_qr_data_url()</code> , S'assurer de la génération, en s'assurant de l'encodage: 'data'. Utilisé pour ne pas avoir à créer de fichier temporaire	La sortie est encodée en base64 et est directement exploitable par le widget <code>ft.Image()</code>	Unitaire
TI-01	Génération HTML	simuler un dossier contenant test.txt et un sous dossier, Envoyer un GET /	le serveur retourne code 200 et des balises <a> vers les fichiers de tests	Intégration
TI-02	Blocage Traversal	simuler une URL inaccessible, comparer <code>requested_path</code> . Avec <code>base_dir</code> grâce à la méthode <code>startswith()</code>	Le serveur retourne HTTP 403 (accès refusé) lorsque le chemin demandé n'est pas le répertoire partagé. Un code 404 si le chemin ne correspond pas à un fichier (aucune route)	Intégration
TI-03	Zip du répertoire	Avec un dossier simulé dans un répertoire partagé, un GET /subfolder appelle <code>shutil.make_archive</code>	le client reçoit un header <code>Content-Disposition: nom_dossier.zip</code> , donc un zip.	Intégration
TE-01	Utilisation smartphone	Activer la protection par mot de passe, lancer le serveur, scanner le QR Code avec un smartphone Android sur le même sous-réseau, télécharger un fichier et un dossier.	la page d'authentification s'affiche, le mot de passe redirige vers la seconde page de téléchargement, les logs affichent les actions en temps réel, le client parvient à télécharger ses fichiers	End-to-End
TP-01	Utilisation utilisateur tiers	Demander à l'utilisateur de partager un fichier test.txt créé ensemble dans le répertoire Documents, et de le récupérer sur son smartphone.	L'utilisateur parvient à naviguer sans gêne sur l'application. L'application semble intuitive à l'utilisateur. Il ne doit pas être frustré de ne pas parvenir à faire une action	Utilisateur

TU-04	Logging persistant (niveau)	Appeler <code>model.log_to_file()</code> avec chaque niveau ('success', 'info', 'warning', 'error') Ouvrir le fichier <code>sharepp.log</code> et vérifier le contenu.	Chaque message est écrit dans le fichier <code>.log</code> avec le bon préfixe de niveau (INFO / WARNING / ERROR) et un horodatage au format attendu.	Unitaire
TU-05	Credentials hotspot validation mdp correct	Appeler <code>hotspot_controller.setup_and_start(ssid, '1234567')</code> avec un mot de passe de moins de 8 caractères.	La fonction retourne (True) sans exécuter netsh.	Unitaire/ Sécurité
TU-06	Credentials hotspot refus mdp court	Appeler <code>hotspot_controller.setup_and_start(ssid, '1234567')</code> avec un mot de passe de moins de 8 caractères.	La fonction retourne (False, 'Le mot de passe doit faire 8 caractères.') sans exécuter netsh.	Unitaire/ Sécurité
TE-02	hotspot ad hoc + partage complet (Windows)	Sur l'ordinateur Windows du CPNV : créer un hotspot via l'interface graphique, connecter un smartphone au réseau créé, démarrer le serveur de fichier, scanner le QR URL et télécharger un fichier.	Le smartphone se connecte au réseau hotspot. Le QR Wi-Fi connecte bien le smartphone au réseau. La page de téléchargement est accessible et le fichier est reçu intact.	End-to-End

5.8 Journal de tests

ID	Cas de test	Résultat	Date d'exécution	Observations
TU-01	Persistence JSON	FAIL	30-04-2026	Problème rencontrés : <code>config_path</code> n'est plus la syntaxe correcte utilisée dans le constructeur. Modification en <code>config_name</code> et utilisation de <code>tmp_path</code> pour effectuer les tests non pas sur les vrais fichiers mais sur des fichiers temporaires Solution trouvée : Réécrire le test afin de modifier le nom de la variable et passer à un répertoire de travail temporaire pour le test.

				Problème rencontrés : config_path n'est plus la syntaxe correcte utilisée dans le constructeur. Modification en config_name et utilisation de tmp_path pour effectuer les tests non pas sur les vrais fichiers mais sur des fichiers temporaires Solution trouvée : Réécrire le test afin de modifier le nom de la variable et passer à un répertoire de travail temporaire pour le test.
TU-02	Hashage password	FAIL	30-04-2026	
				Problème rencontré : la logique de generation de QR code avait été déplacé du model au contrôleur durant le dernier projet sans modifier le test accordément. Solution trouvée : Il faut réécrire les imports, déplacer le test de fichier (test_model.py -> test_controller.py)
TU-03	Génération QR	FAIL	30-04-2026	
TU-04	Logs persistent	PASS	30-04-2026	
				Attention, le test est passé, mais la validation de la fonctionnalité n'a pas encore pu être testée de manière pratique. (absence de carte Wi-Fi à ce moment-ci du projet) Il faudra peut-être adapter le test plus tard
TU-05	Création Access Point	PASS	30-04-2026	
TI-01	Génération HTML	PASS	30-04-2026	
TI-02	Blocage Traversal	PASS	30-04-2026	
TI-03	ZIP répertoire	PASS	30-04-2026	
	Utilisation			
TE-01	smartphone	PASS	30-04-2026	
TU-01	Persistence JSON	PASS	30-04-2026	Nouveau test implémenté et validé
TU-02	Hashage password	PASS	30-04-2026	Nouveau test implémenté et validé
				Nouveau test implémenté, validé et déplacé
TU-03	Génération QR	PASS	30-04-2026	
TU-01	Persistence JSON	PASS	18-05-2026	
TU-02	Hashage password	PASS	18-05-2026	
TU-03	Génération QR	PASS	18-05-2026	
TU-04	Logs persistent	PASS	18-05-2026	
	Création Access			
TU-05	Point	PASS	18-05-2026	La création de l'access point a été validé sur le PC Windows du CPNV.
TI-01	Génération HTML	PASS	18-05-2026	
TI-02	Blocage Traversal	PASS	18-05-2026	
TI-03	ZIP répertoire	PASS	18-05-2026	
	Utilisation			
TE-01	smartphone	PASS	18-05-2026	

TE-02	Création hotspot ad hoc + partage complet	FAIL	18-05-2026	La validation finale depuis l'ordinateur du CPNV reste bloqué, Il peut s'agir d'une restriction du réseau CPNV.
TP-01	Utilisation utilisateur tiers	PASS	18-05-2026	L'utilisateur a su utiliser l'application de manière plus intuitive que la fois passée. La mise en évidence du bouton de démarrage/arrêt a bien été reçu. L'utilisateur a encore une fois trouvé étrange de partager un répertoire plutôt qu'un fichier. Aucun message d'avertissement n'est apparu durant le processus, le fichier a bien été réceptionné

5.9 Archives du projet

L'intégralité des fichiers de code sources, l'historique des releases, des exécutions de workflows et de commits sont disponibles sur le dépôt public du projet : github.com/NathanFilipowicz/Sharepp

